CSE 4303
Data Structures

Topic: HeapSort, Priority Queues

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

(Recap)

```
build_max_heap(){
    for i=heapSize/2 downto 1
        max_heapify(i)
}
```

Time complexity?

$$O\left(\frac{N}{2}logN\right) \sim O(NlogN)$$

(Recap)

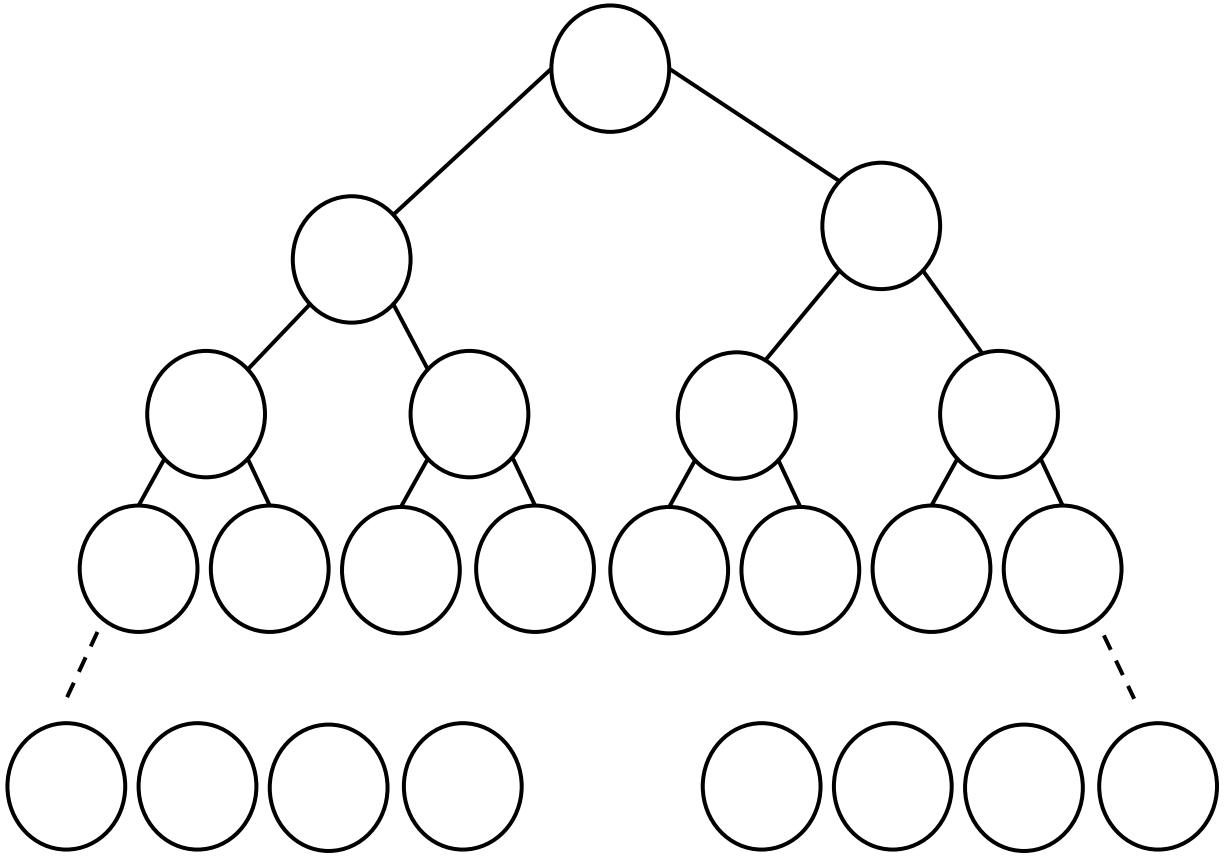
```
build_max_heap(){
    for i=heapSize/2 downto 1
        max_heapify(i)
}
```

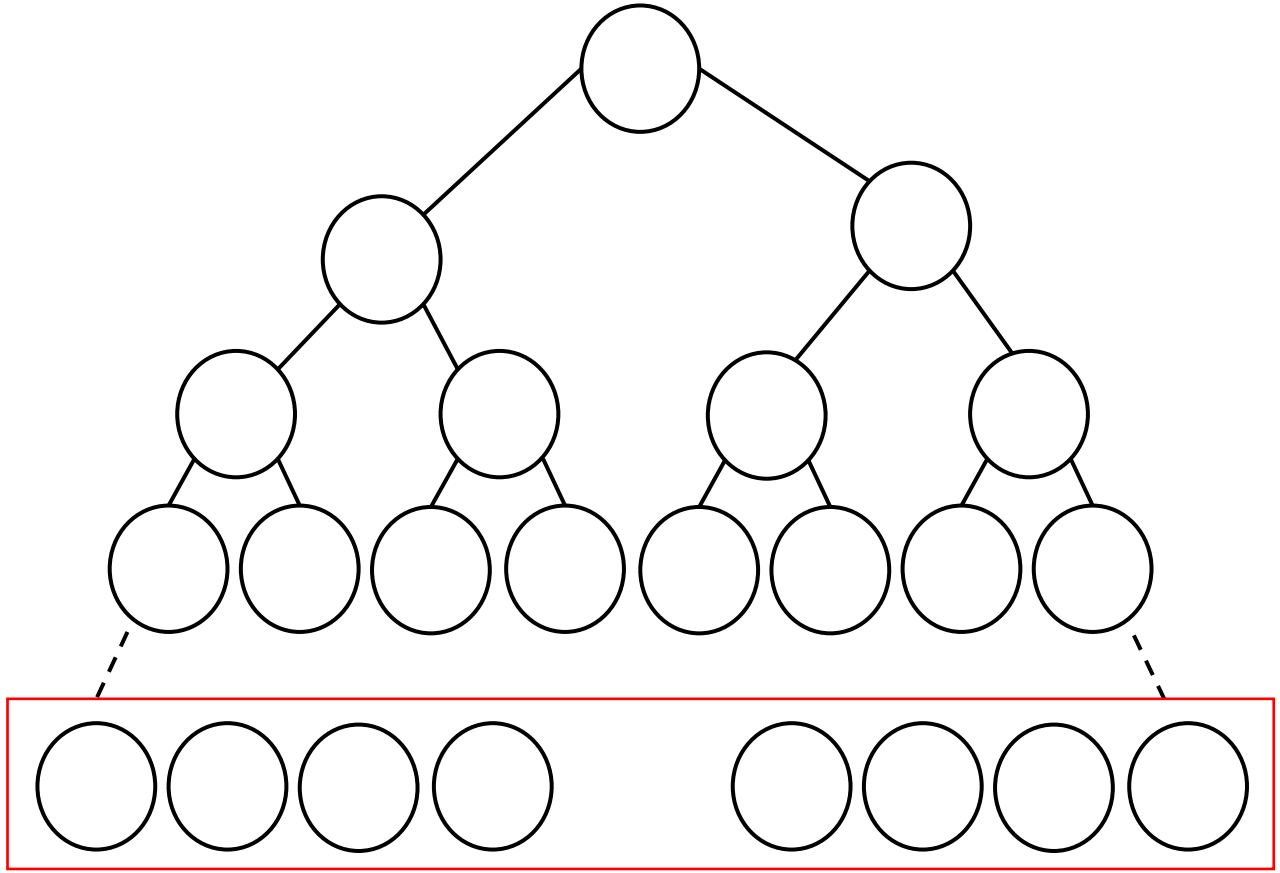
It's a rough estimation.
Detailed analysis shows that the complexity is $O(N)$.
Next lecture!

Time complexity?

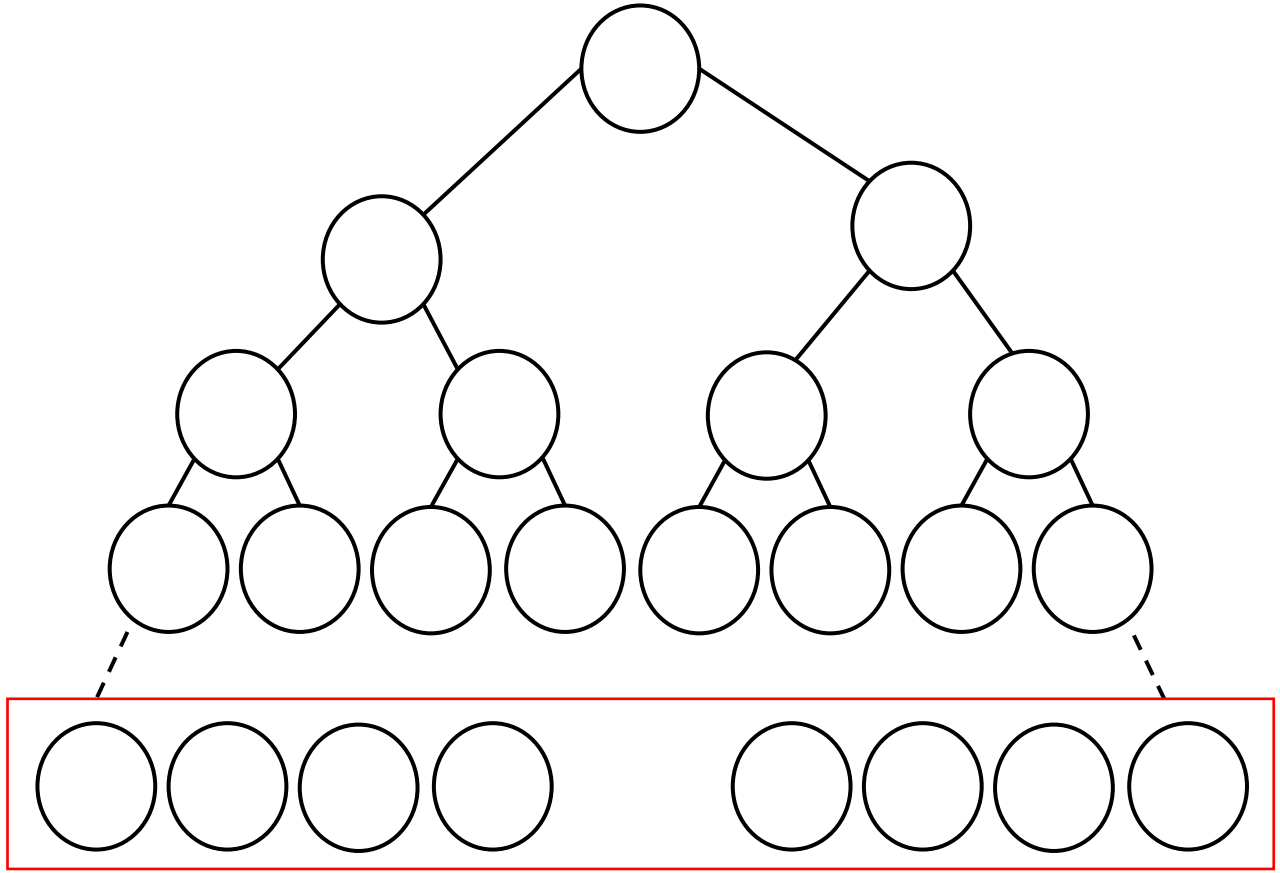
$$O\left(\frac{N}{2}logN\right) \sim O(NlogN)$$

rough estimation!!!





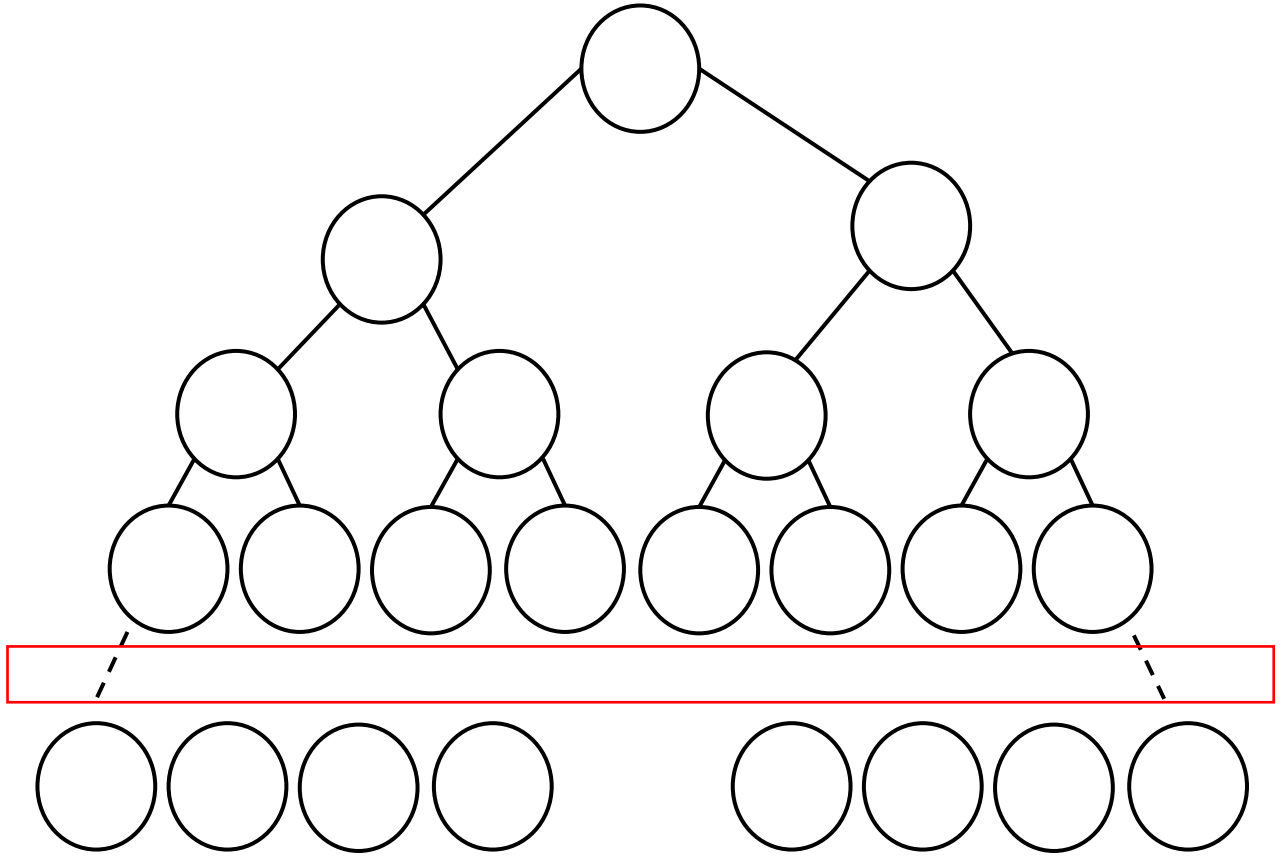
How many nodes can there be in this level?



How many nodes can there be in this level?

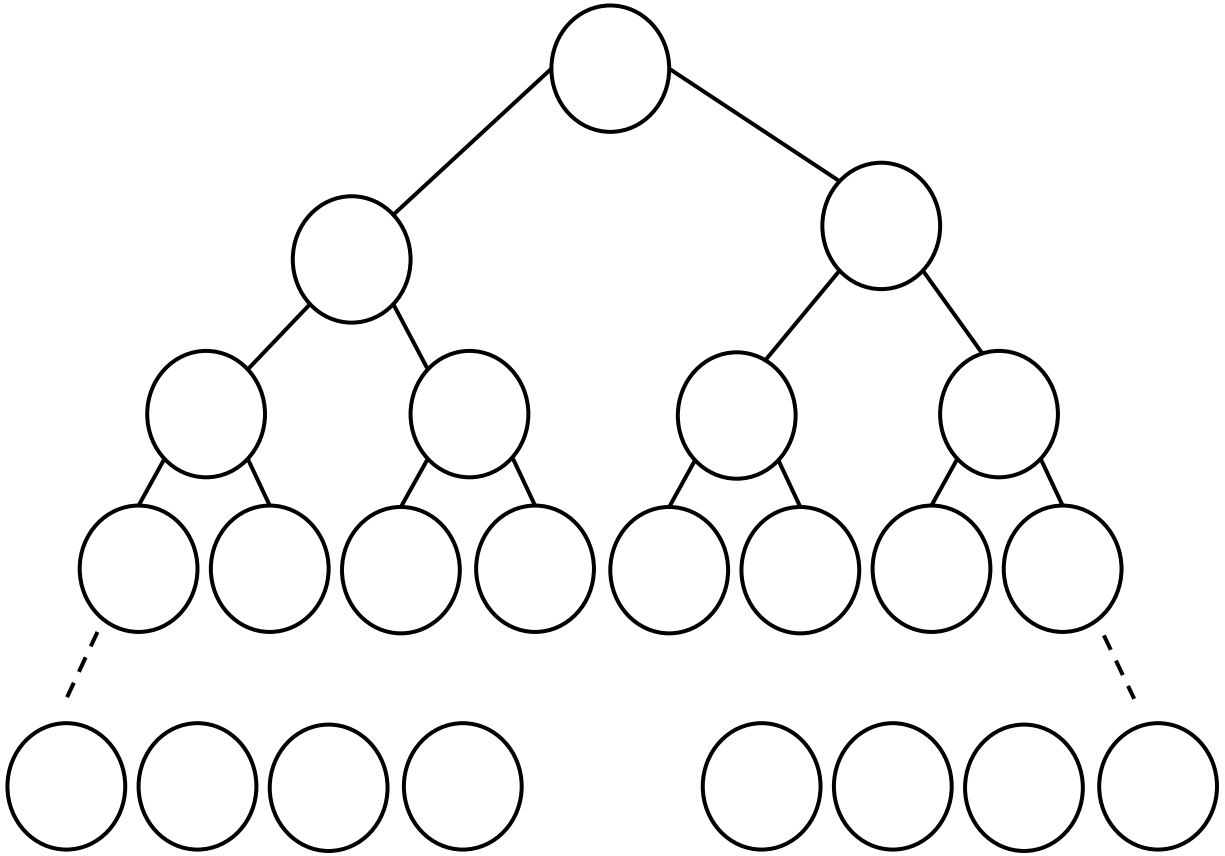
$N/_2$ nodes

How many nodes can there be in this level?



$N/_4$ nodes

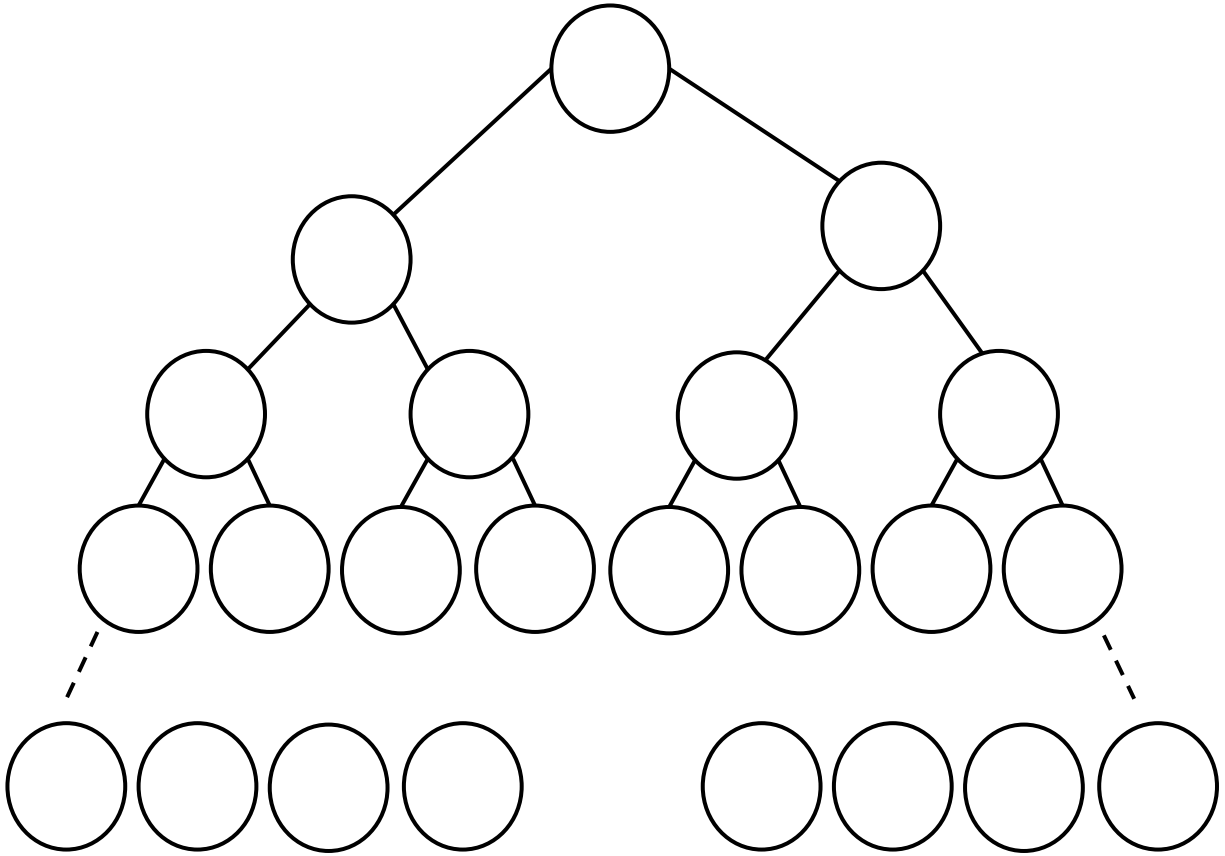
$N/_2$ nodes



1 node

$N/_4$ nodes

$N/_2$ nodes

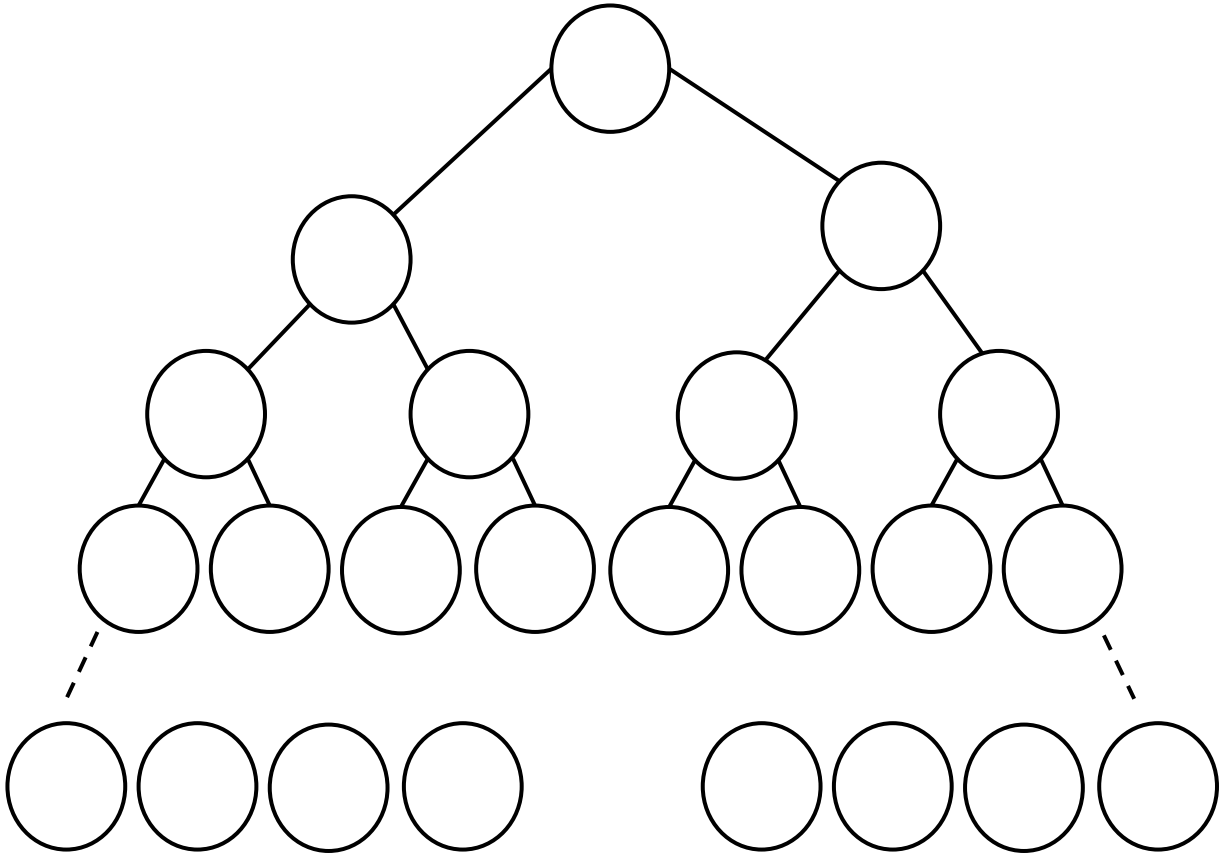


1 node

$N/_4$ nodes

$N/_2$ nodes	0

What is the cost for each node?

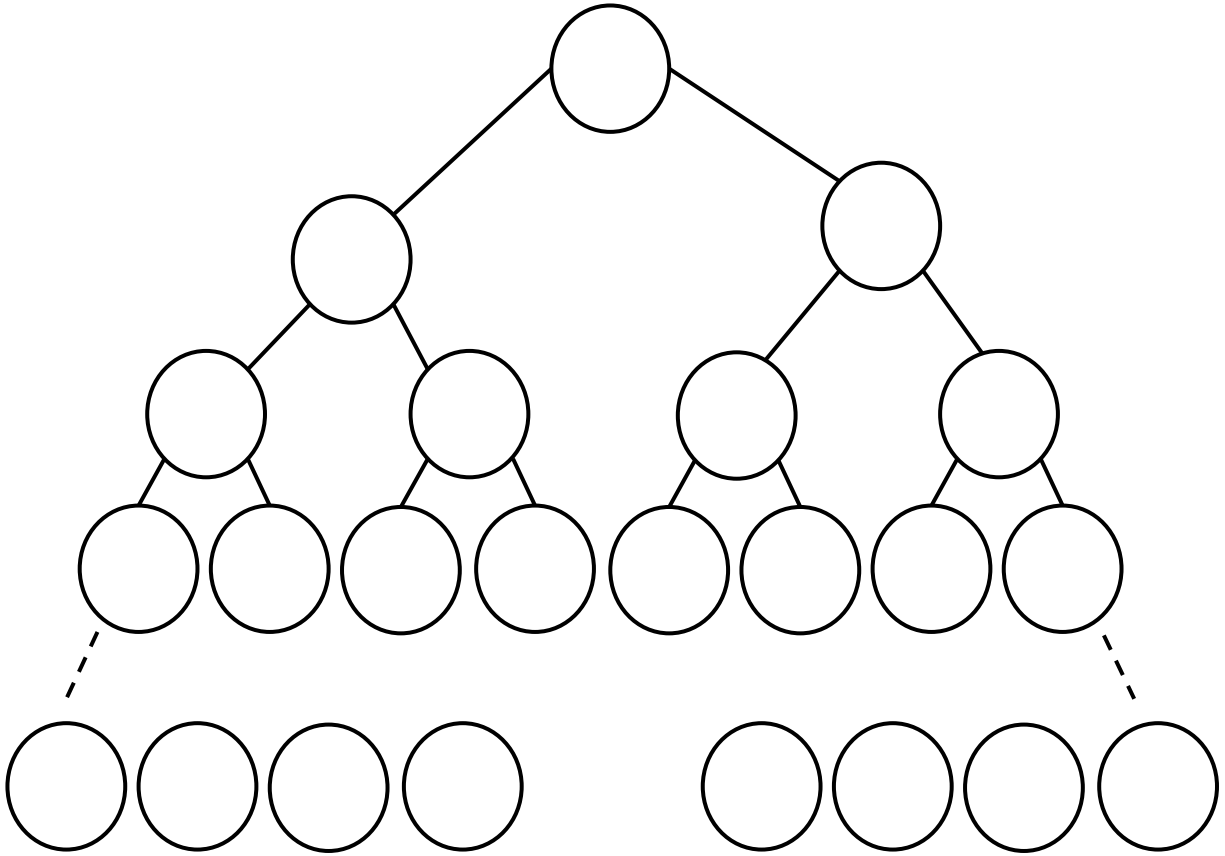


1 node

Each node of 2nd last level requires max 1 operation per node with c time-cost.

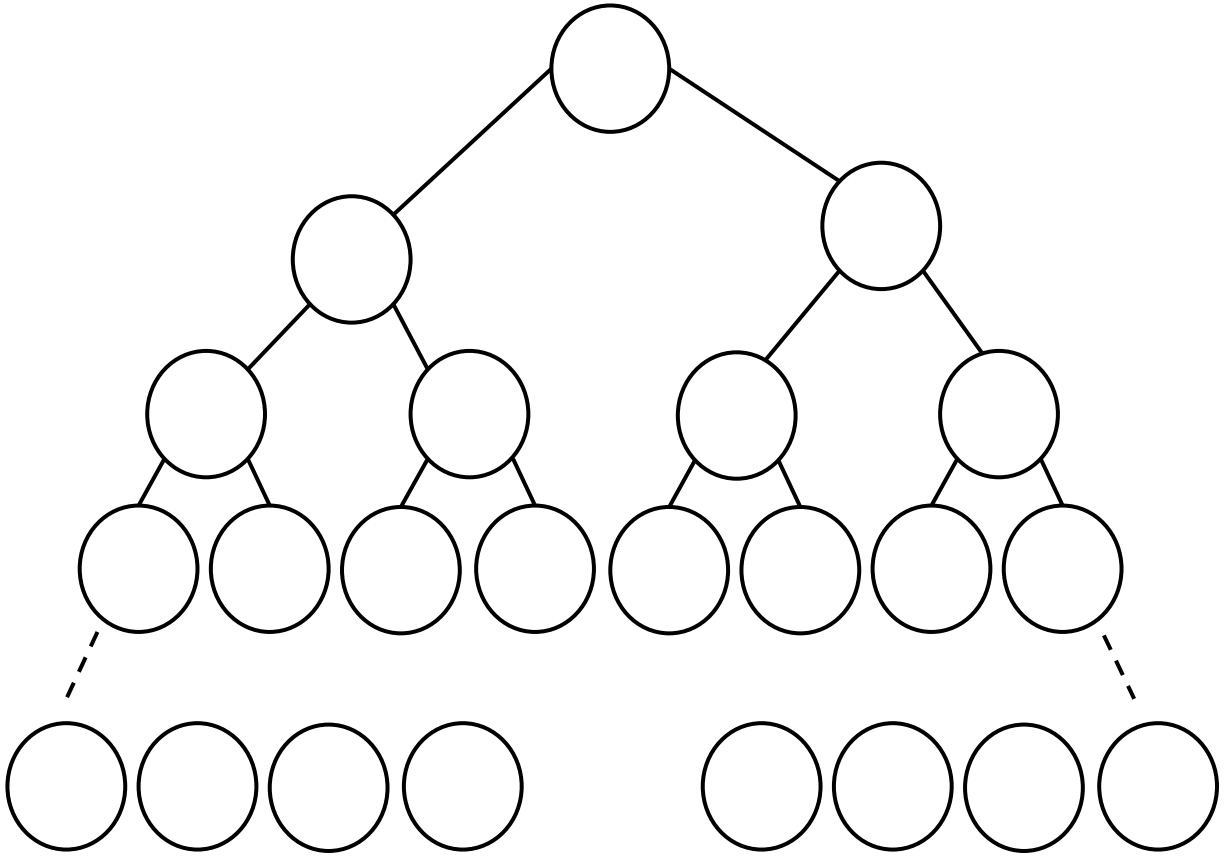
$N/4$ nodes	$1c$
$N/2$ nodes	0

What is the cost for each node?



1 node	$logN$
$N/_4$ nodes	$1c$
$N/_2$ nodes	0

What is the cost for each node?



1 node	$logN$
$N/_4$ nodes	$1c$
$N/_2$ nodes	0

The cost increases as we go towards the root, but only one node requires $logN$ time!

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

$$= 2^k \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{k+1}{2^k} \right)$$



The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}} \right)$$

$$= 2^k \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{k+1}{2^k} \right)$$

This is a decreasing series!

$1 + 1 + 0.75 + 0.5 \ldots \ldots \leq 4 \sim c'$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= 2^k \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{k+1}{2^k}\right)$$

$$= 2^k \times c \times c'$$

$$= O(2^k) = O\left(\frac{N}{4}\right) \sim O(N)$$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= 2^k \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{k+1}{2^k}\right)$$

$$= 2^k \times c \times c'$$

$$= O(2^k) = O\left(\frac{N}{4}\right) \sim O(N)$$

The time complexity of the Build max heap algorithm: $T(N)$

$$= {}^{N}/_{2} \times 0 + {}^{N}/_{4} \times 1c + {}^{N}/_{8} \times 2c + \cdots + 1 \times logN \times c$$

$$= {}^{N}/_{4} \times c \left(1 + {}^{1}/_{2} \times 2 + {}^{1}/_{4} \times 3 + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= {}^{N}/_{4} \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{logN}{{}^{N}/_{4}}\right)$$

$$= 2^k \times c \left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots + \frac{k+1}{2^k}\right)$$

$$= 2^k \times c \times c'$$

$$= O(2^k) \boxed{= O\left(\frac{N}{4}\right) \sim O(N)}$$

Meme alert!

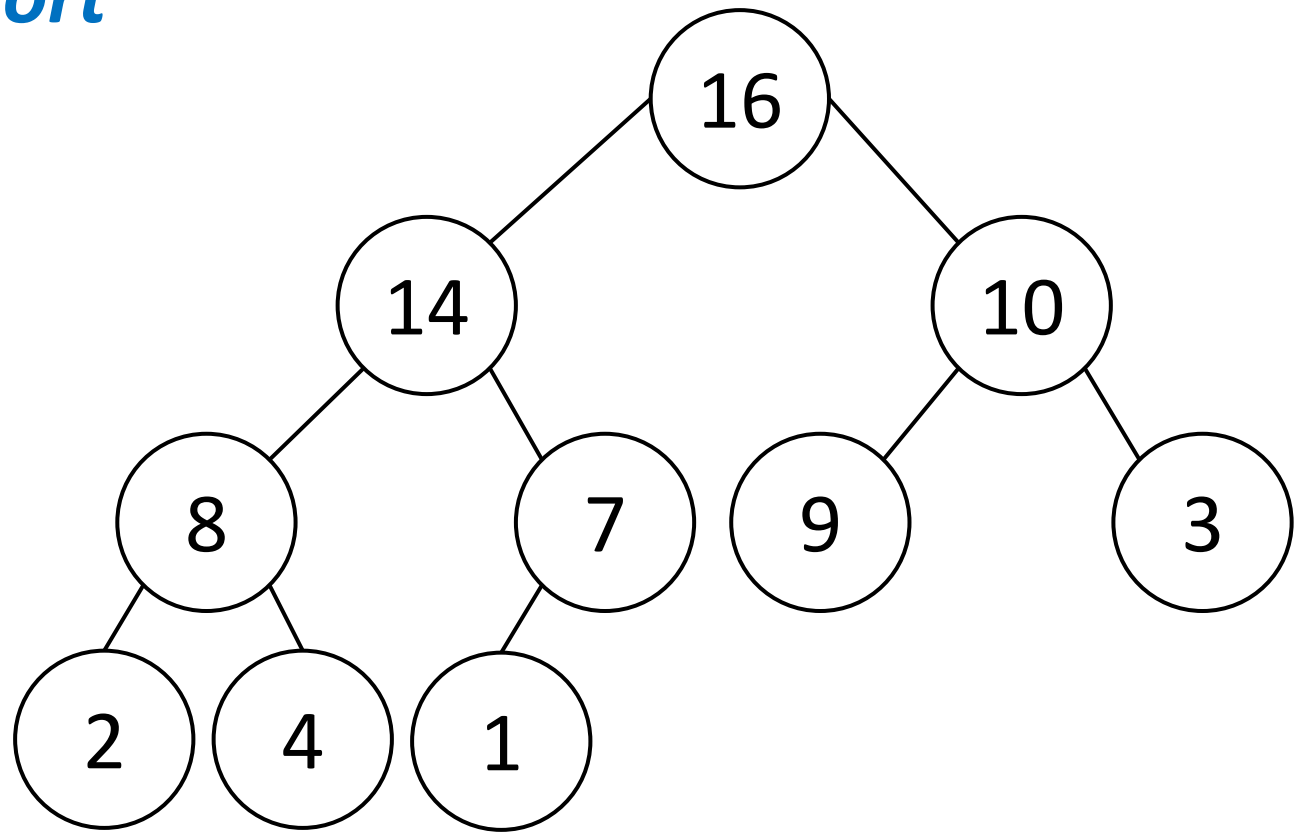

Developer
Learn Any Programming
Language 100%


Developer
Learn Any Programming
Language 100%


Developer
Algorithms ,DataStructures

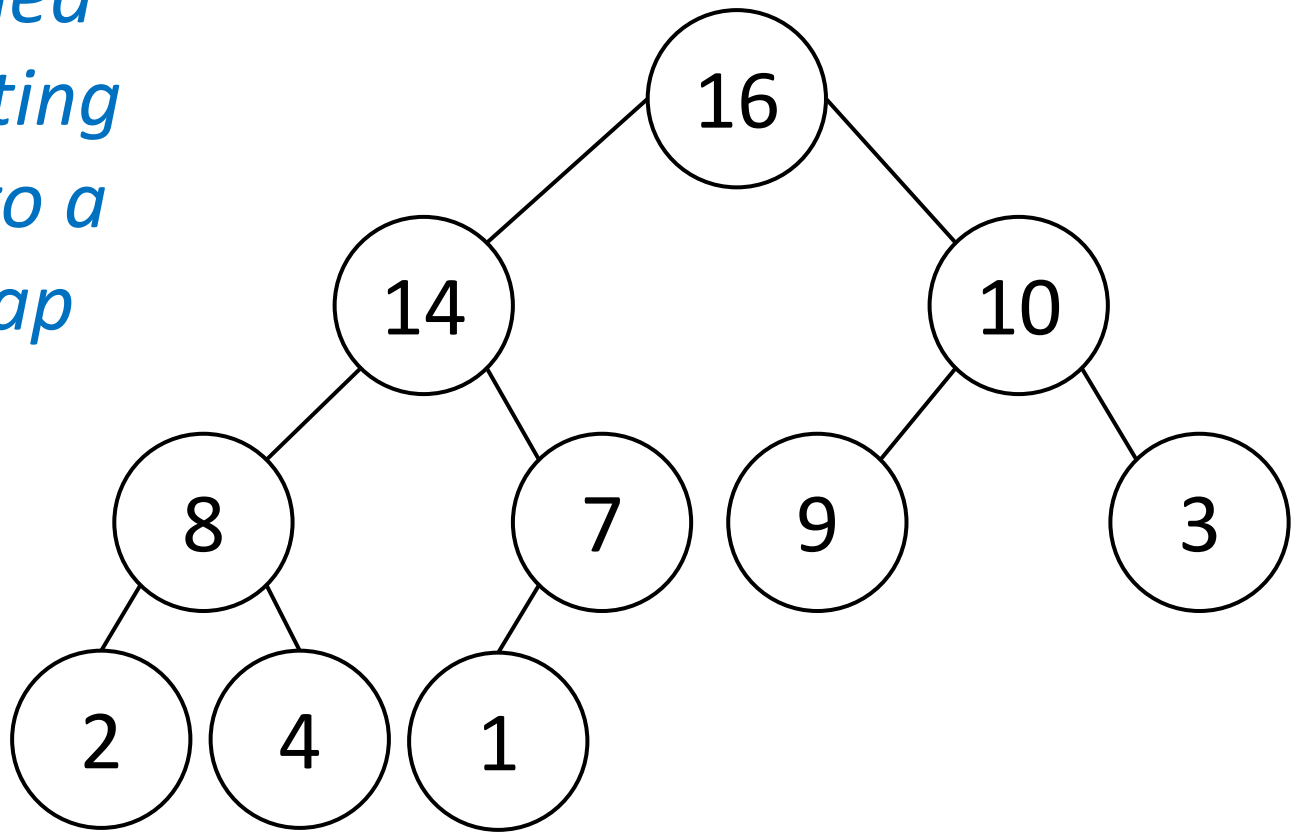
HeapSort

*Sort the numbers using **HeapSort** algorithm*



16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

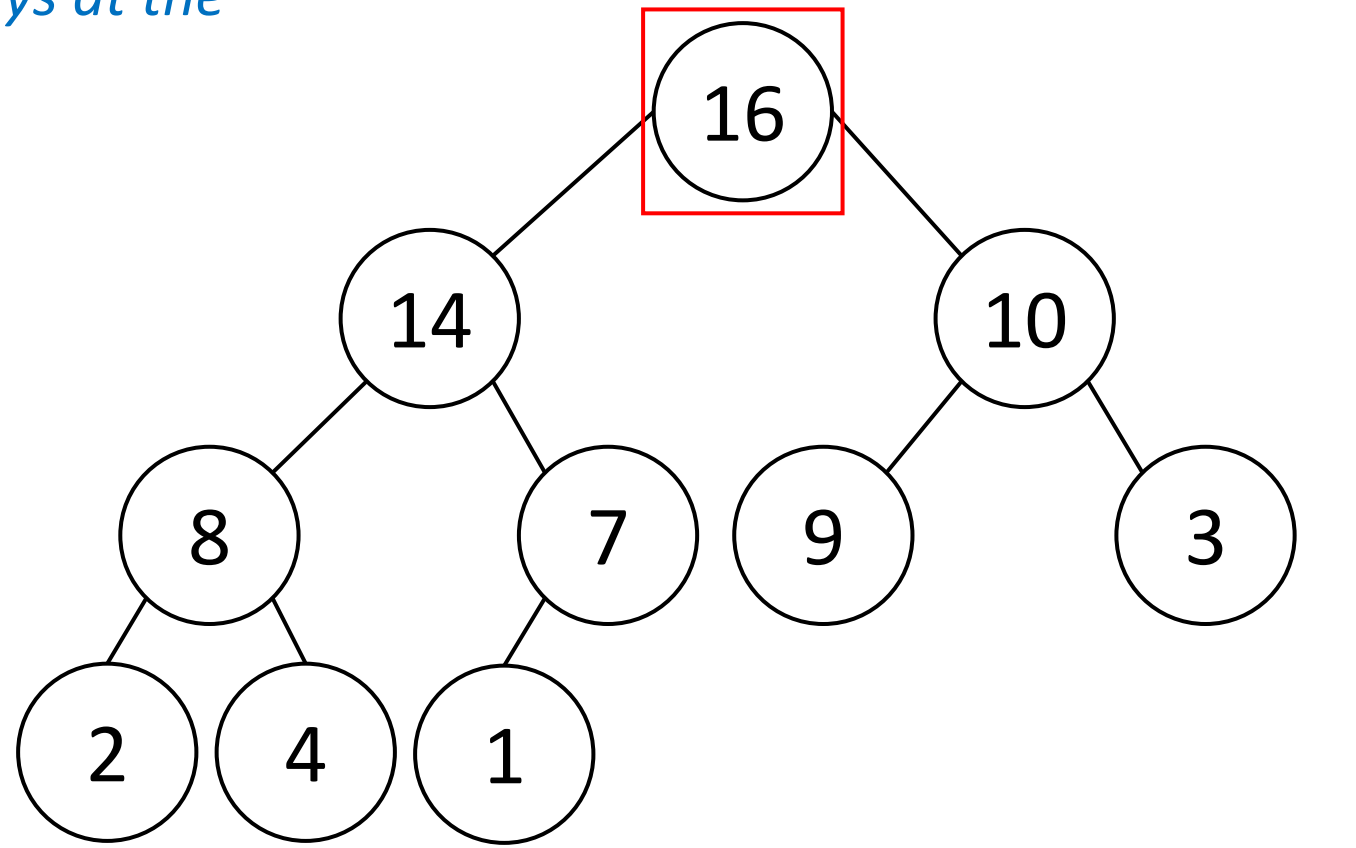
***HeapSort** can only be applied after converting the items into a max/min heap*



16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

Idea:

max/min is always at the root

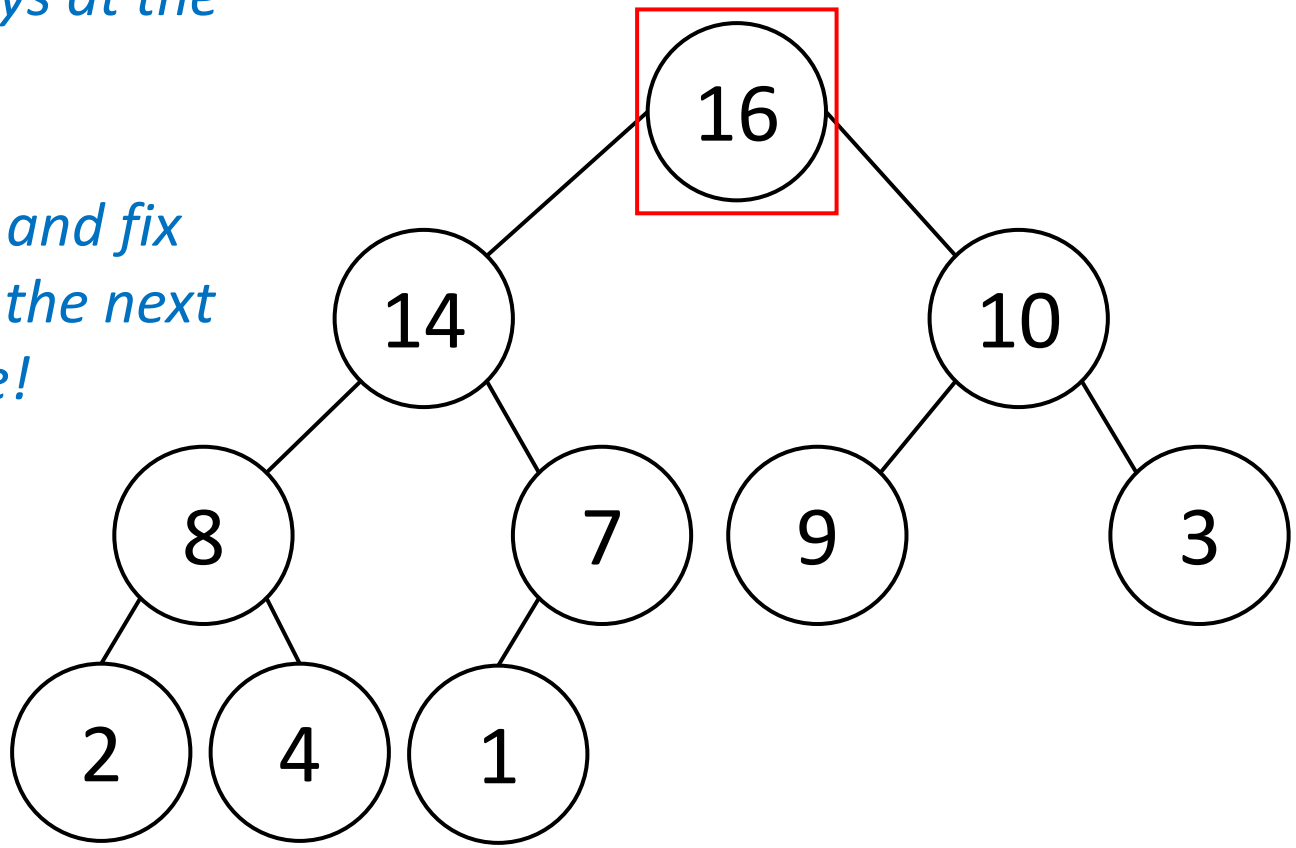


16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

Idea:

max/min is always at the root

Extract the root, and fix that location, so the next max/min is there!



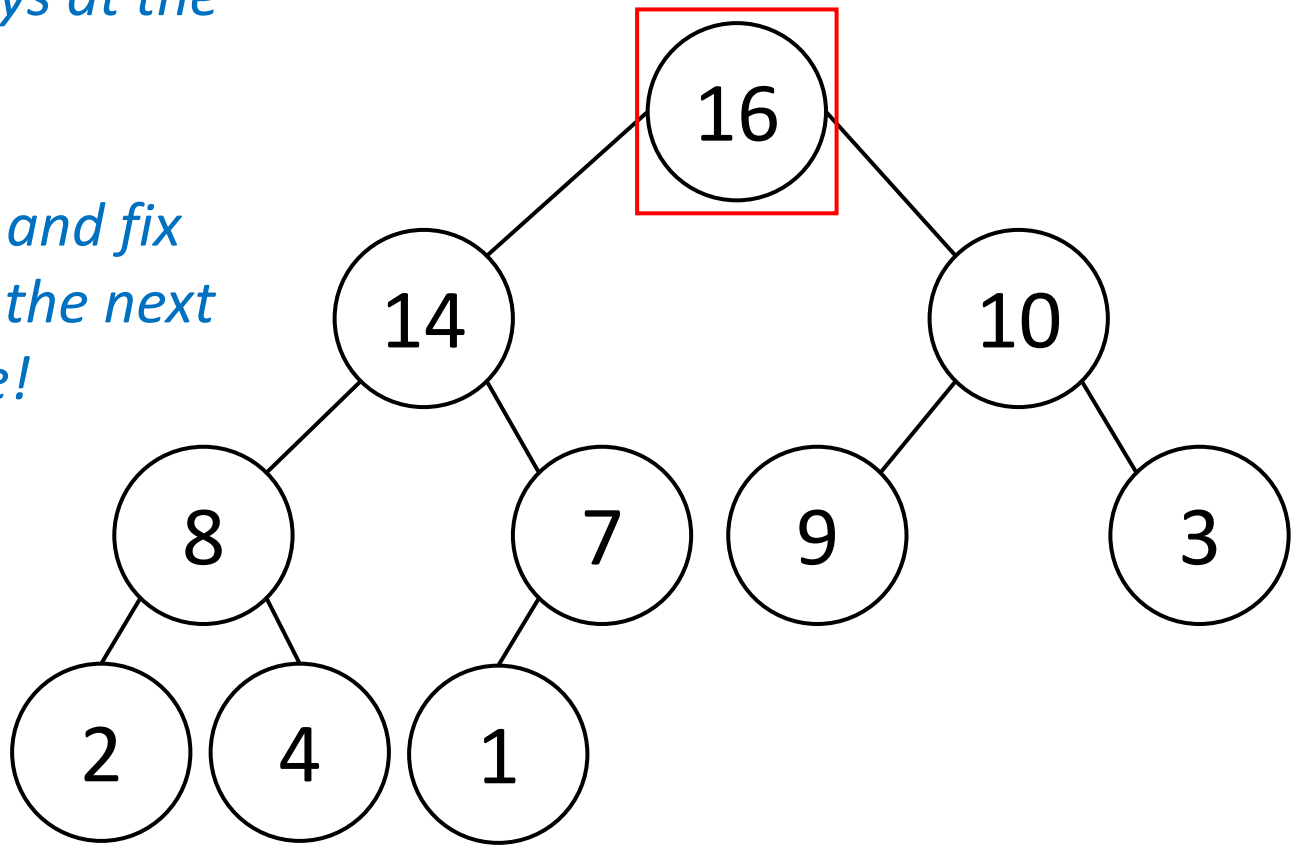
16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

Idea:

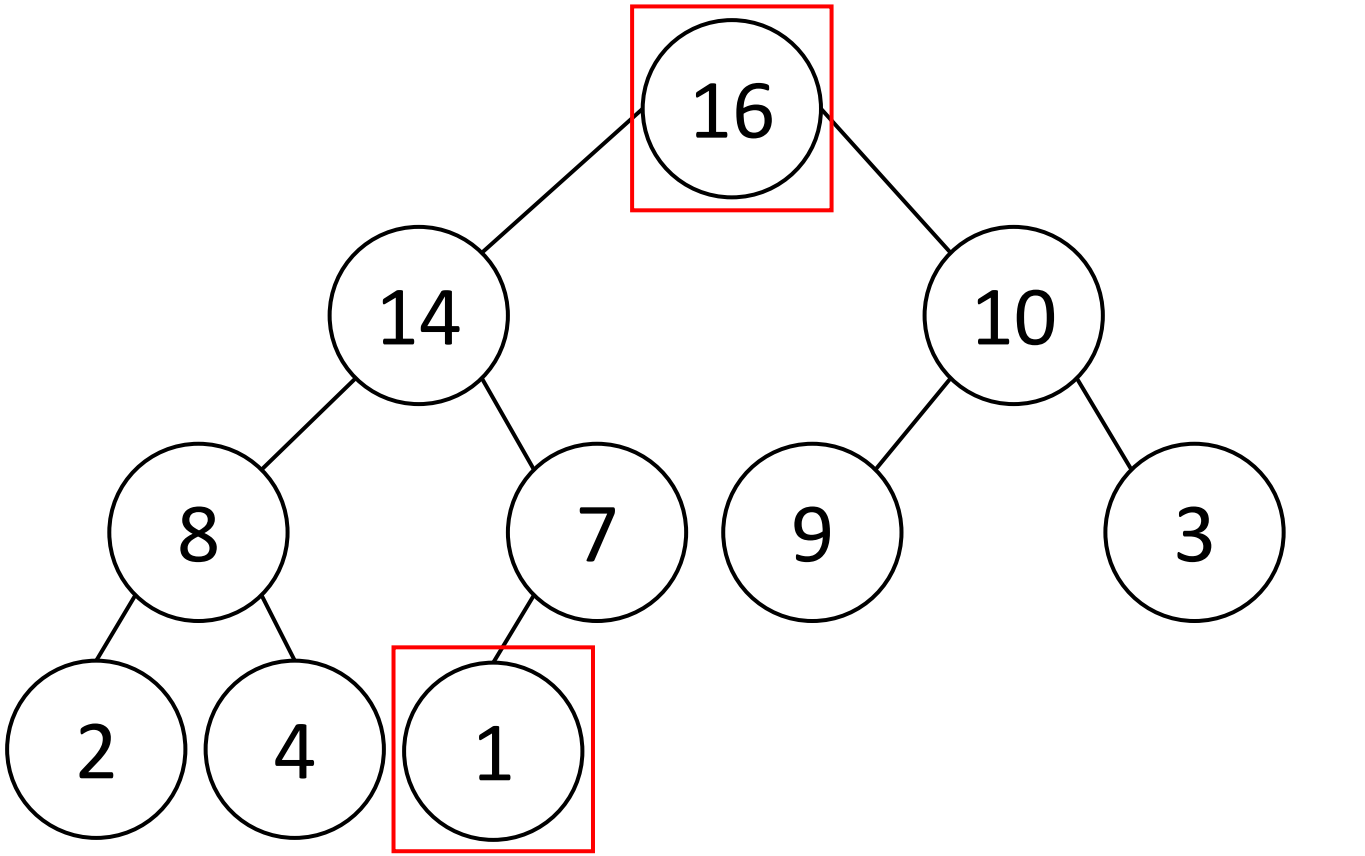
max/min is always at the root

Extract the root, and fix that location, so the next max/min is there!

repeat!

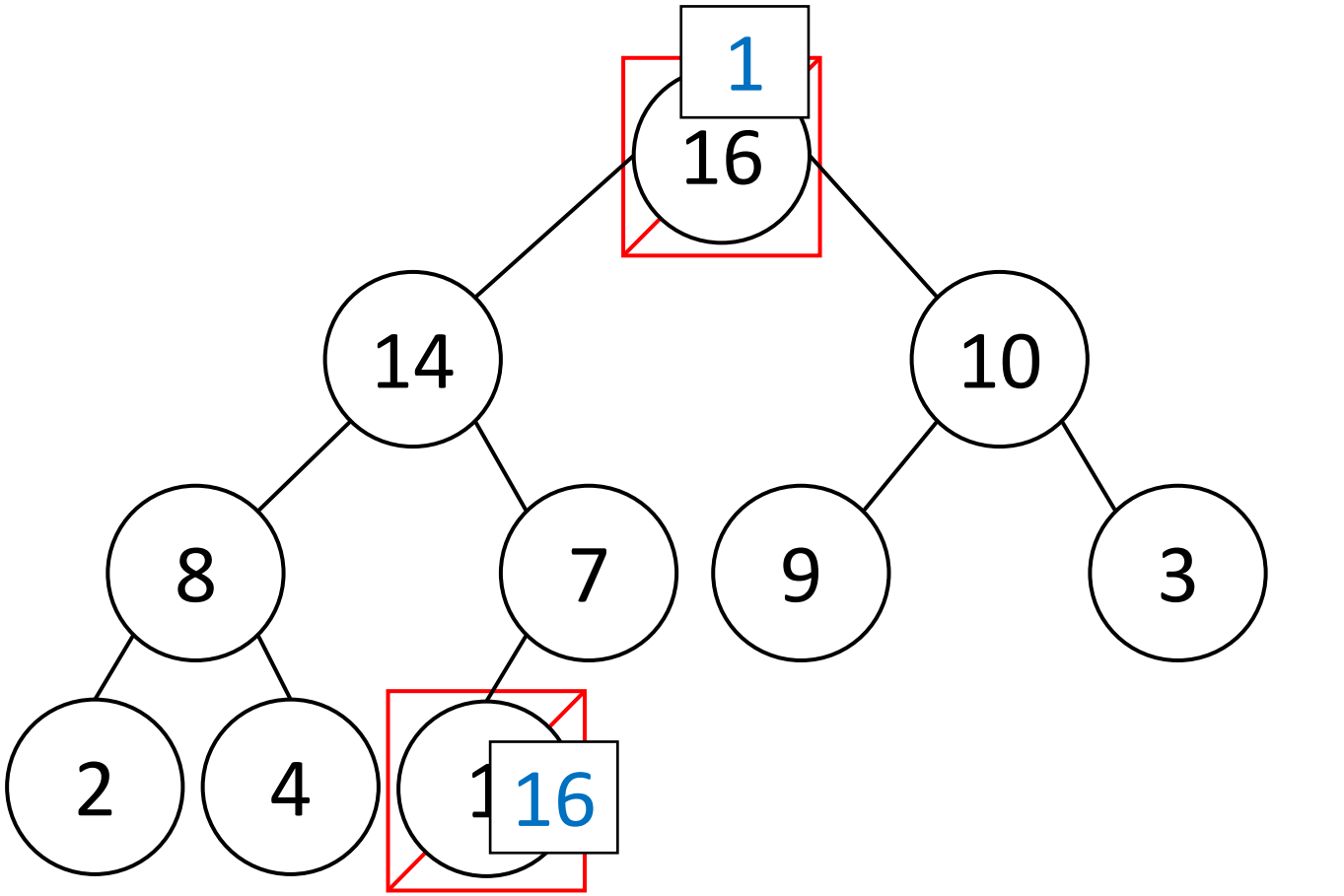


16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10



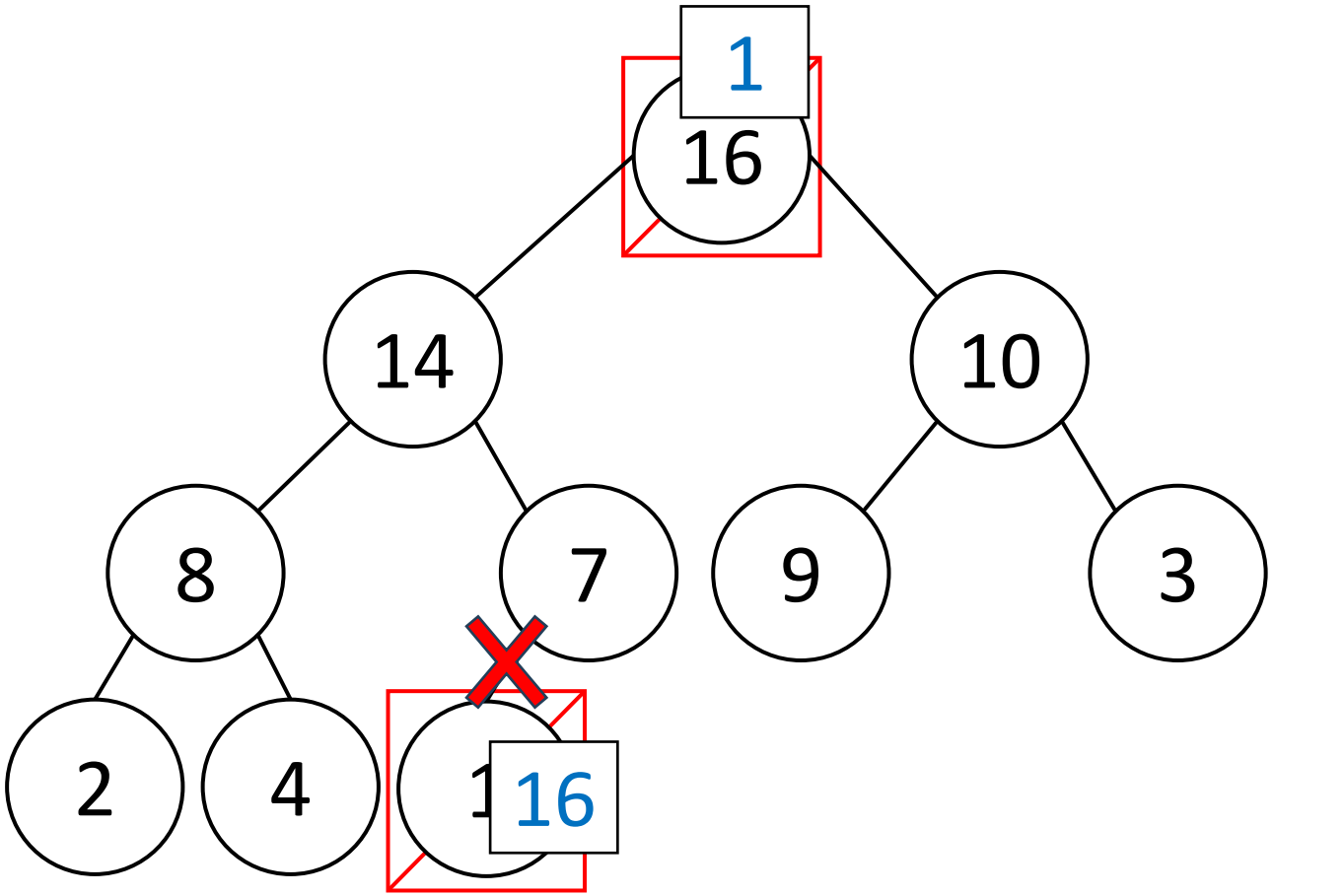
16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

1. Swap root and last location



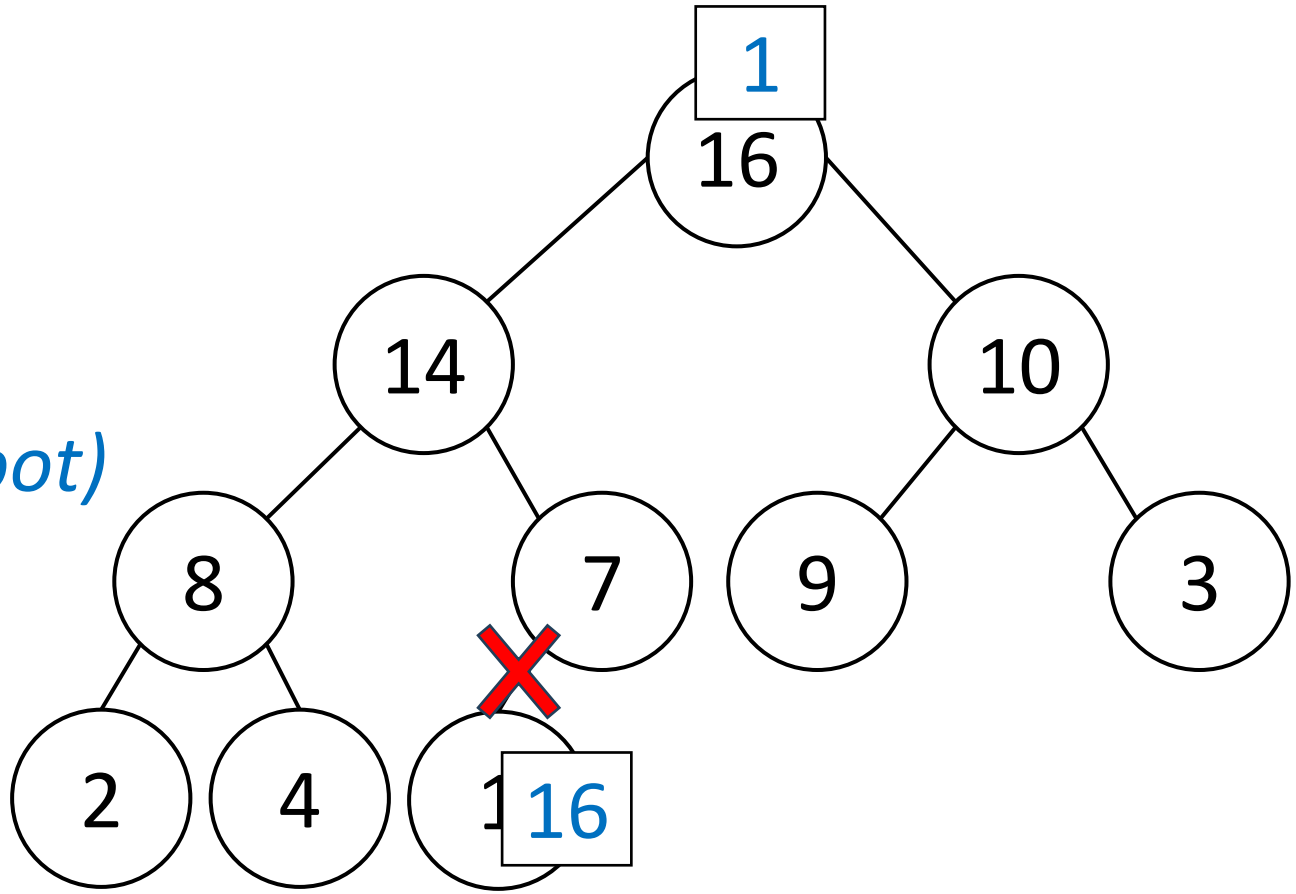
16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

1. *Swap root and last location*
2. *Reduce heapSize*

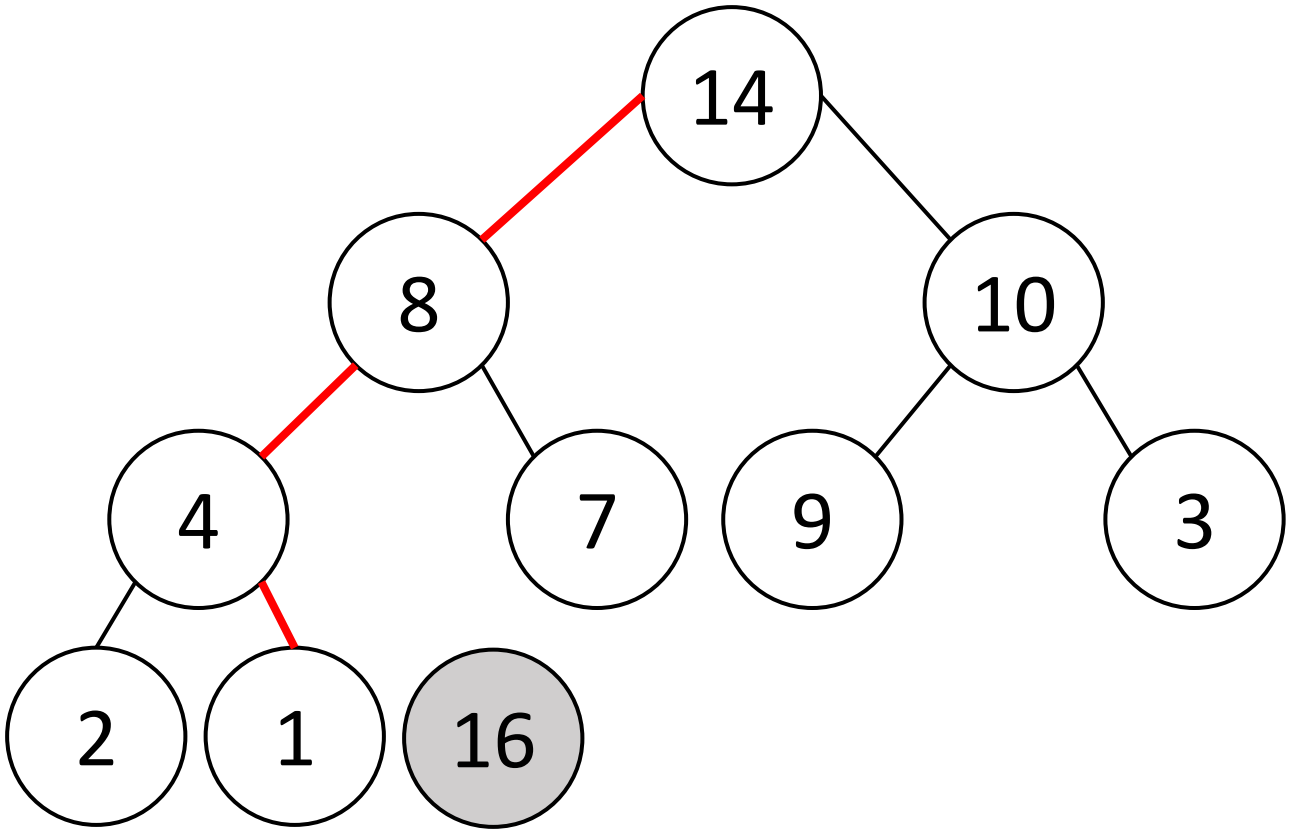


16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

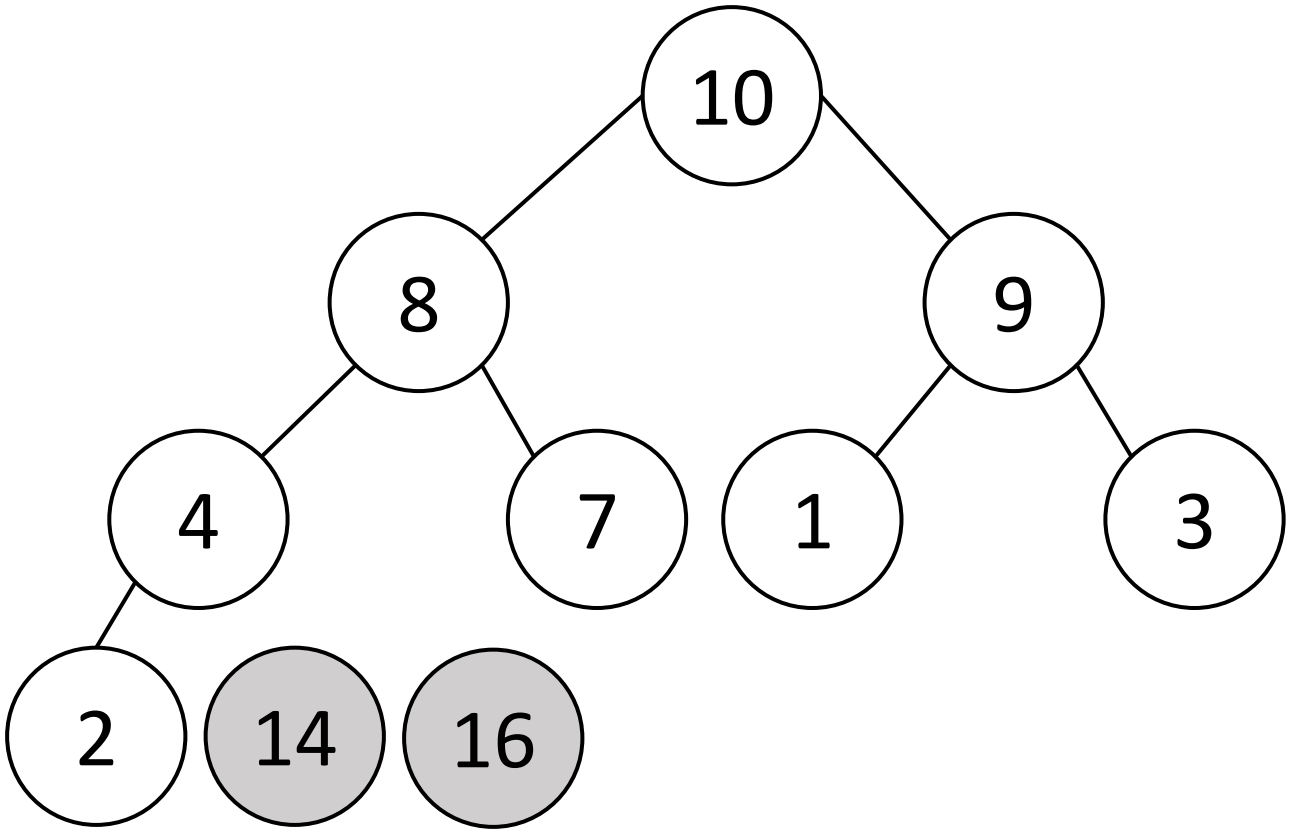
1. *Swap root and last location*
2. *Reduce heapSize*
3. *heapify(root)*



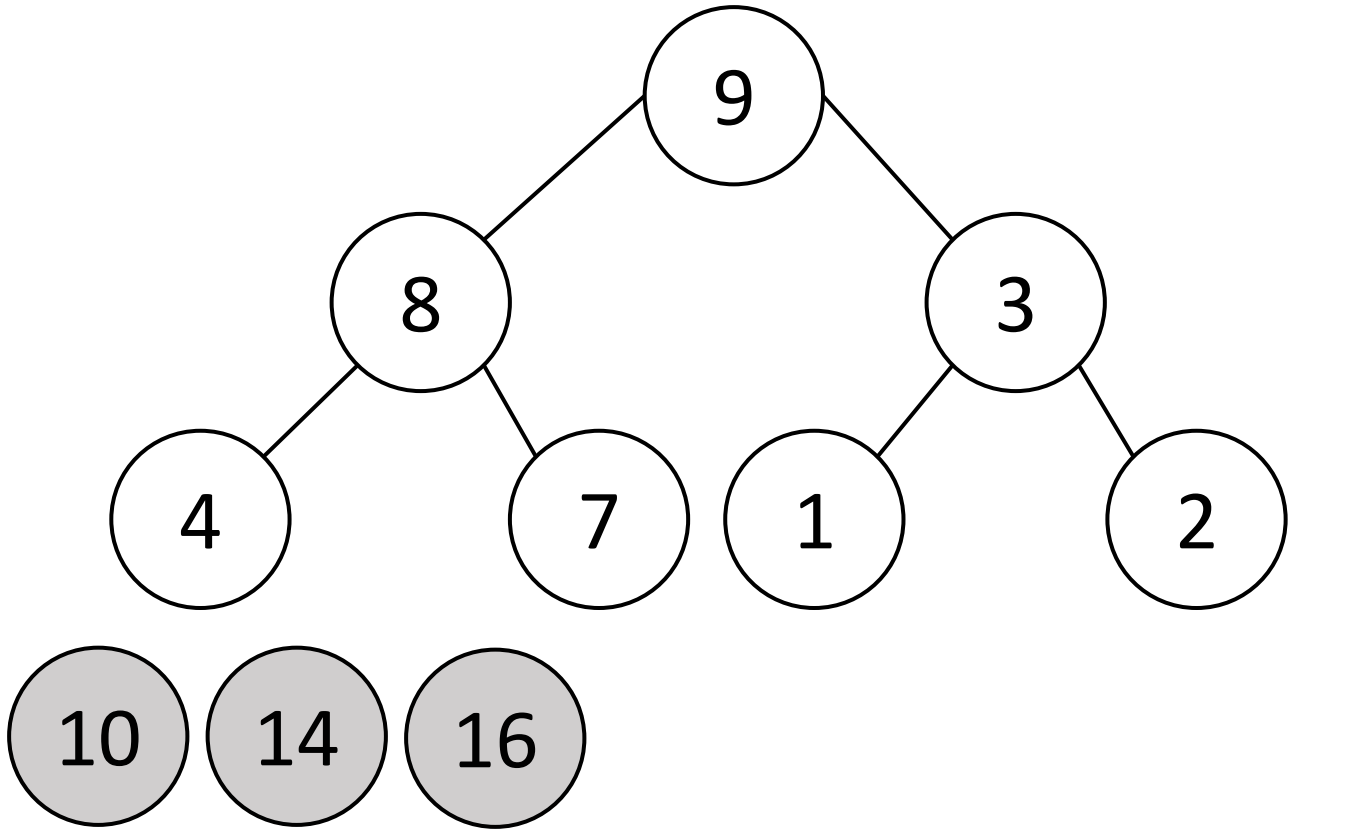
16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10



14	8	10	4	7	9	3	2	1	16
1	2	3	4	5	6	7	8	9	10

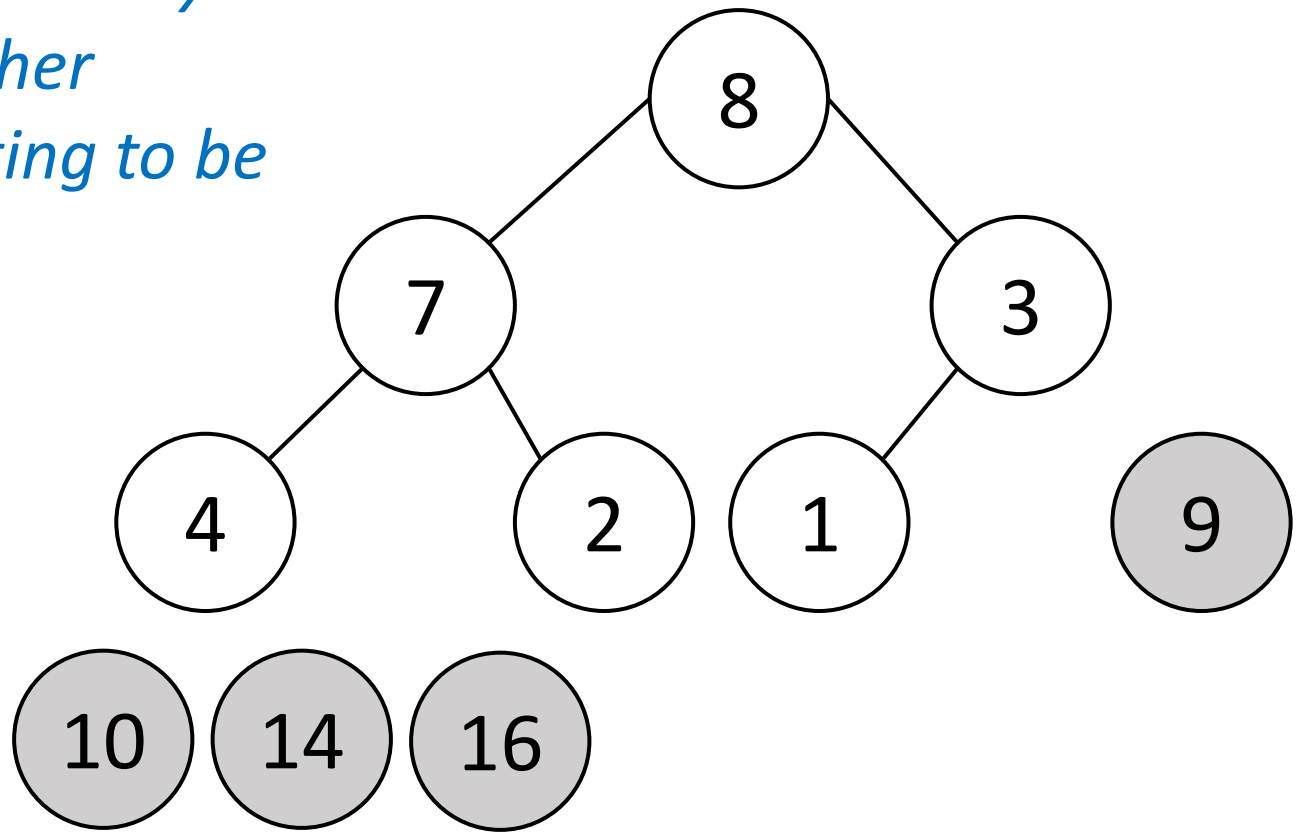


10	8	9	4	7	1	3	2	14	16
1	2	3	4	5	6	7	8	9	10

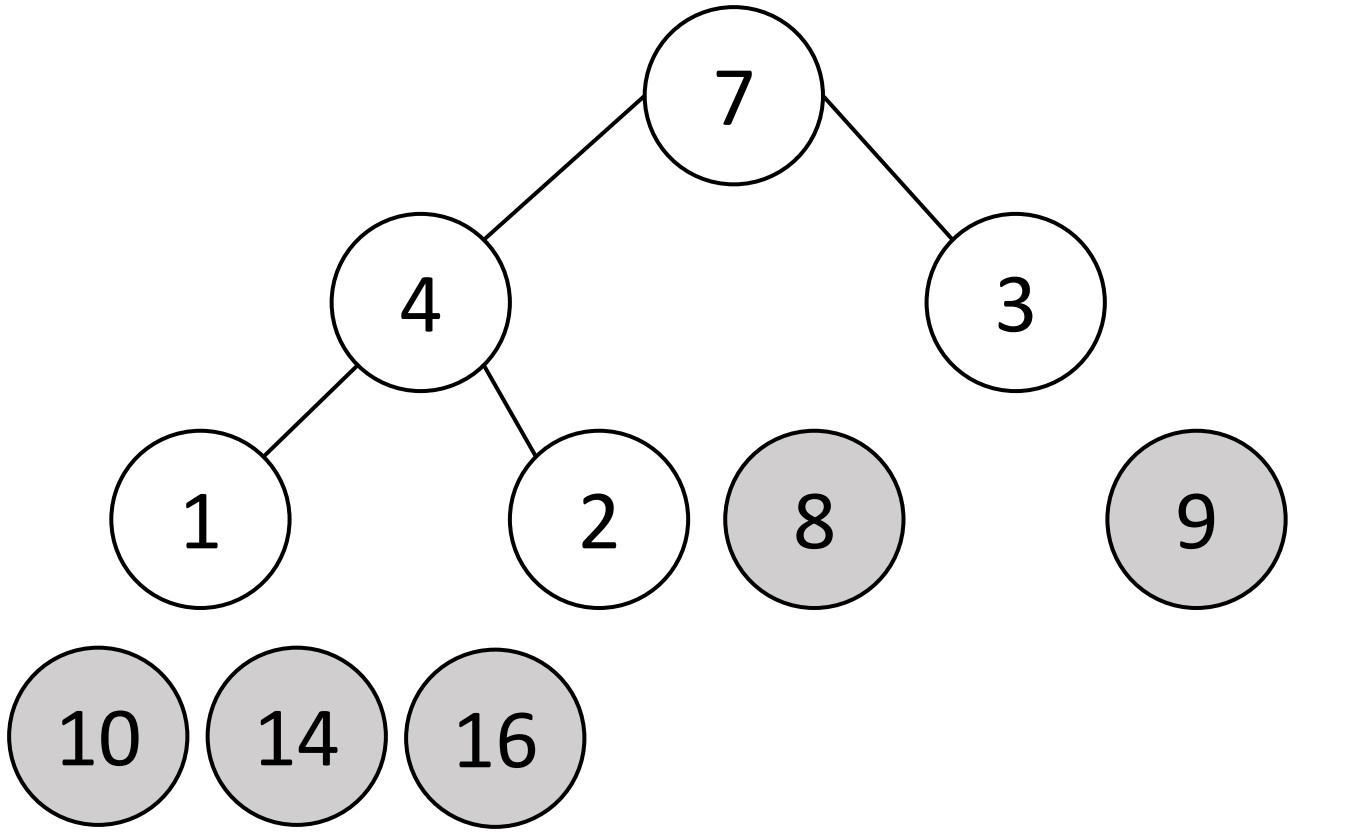


9	8	3	4	7	1	2	10	14	16
1	2	3	4	5	6	7	8	9	10

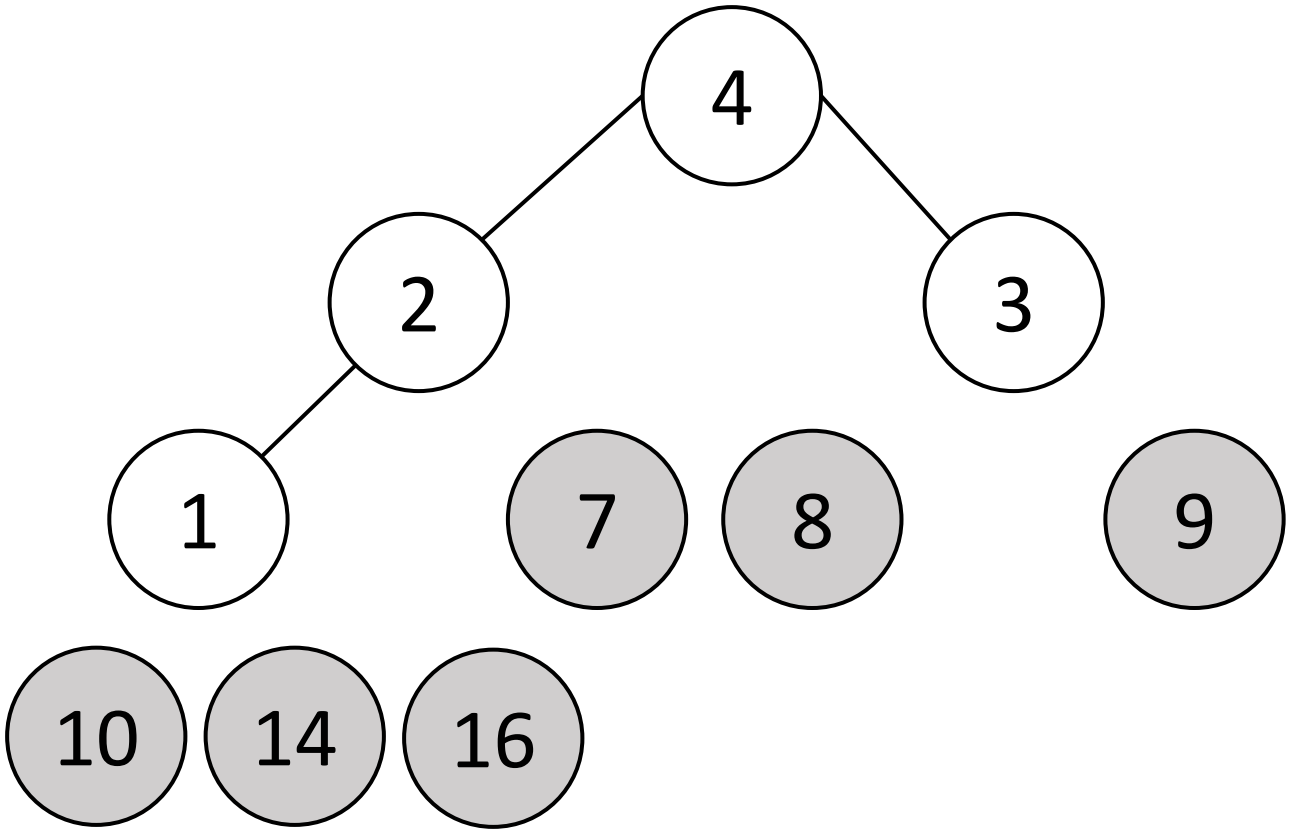
After each step:
One part of the array is sorted, and other items are waiting to be sorted.



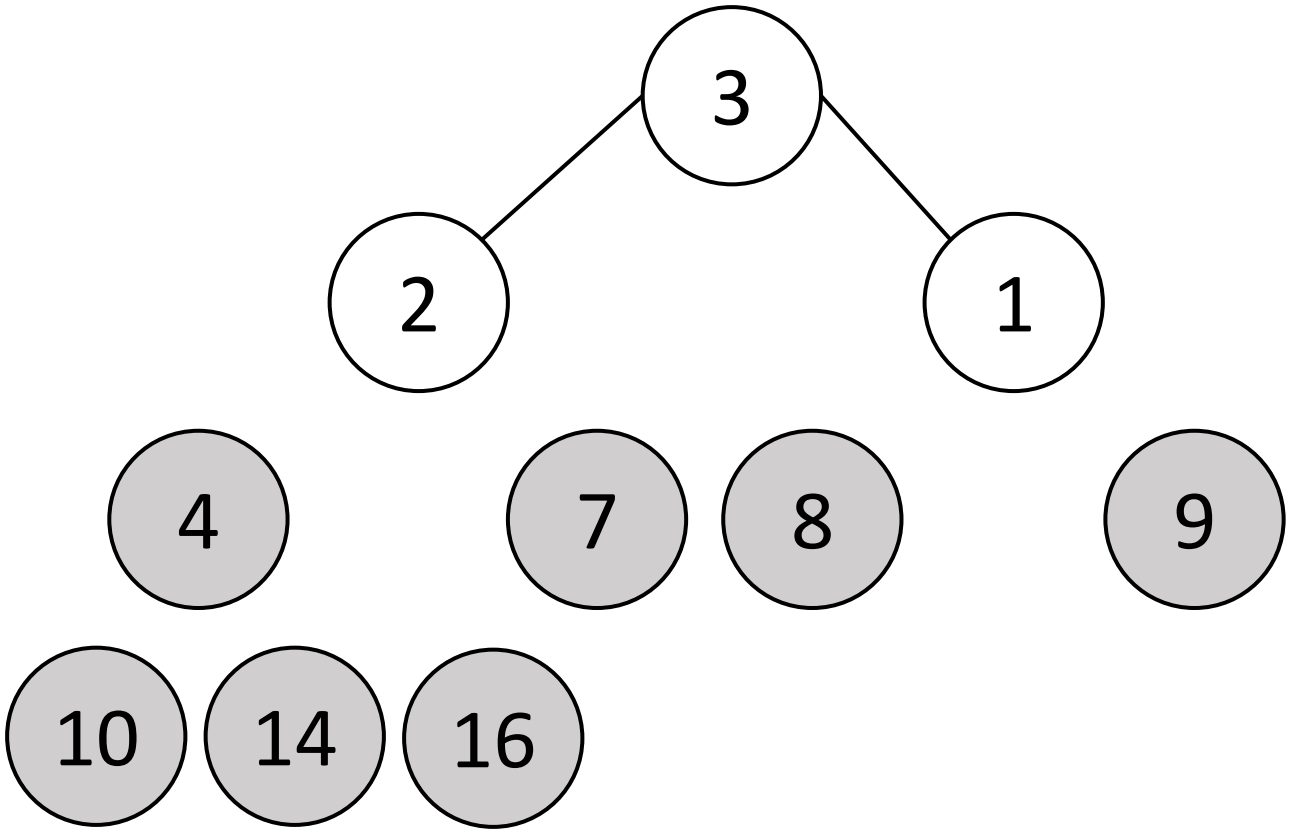
8	7	3	4	2	1	9	10	14	16
1	2	3	4	5	6	7	8	9	10



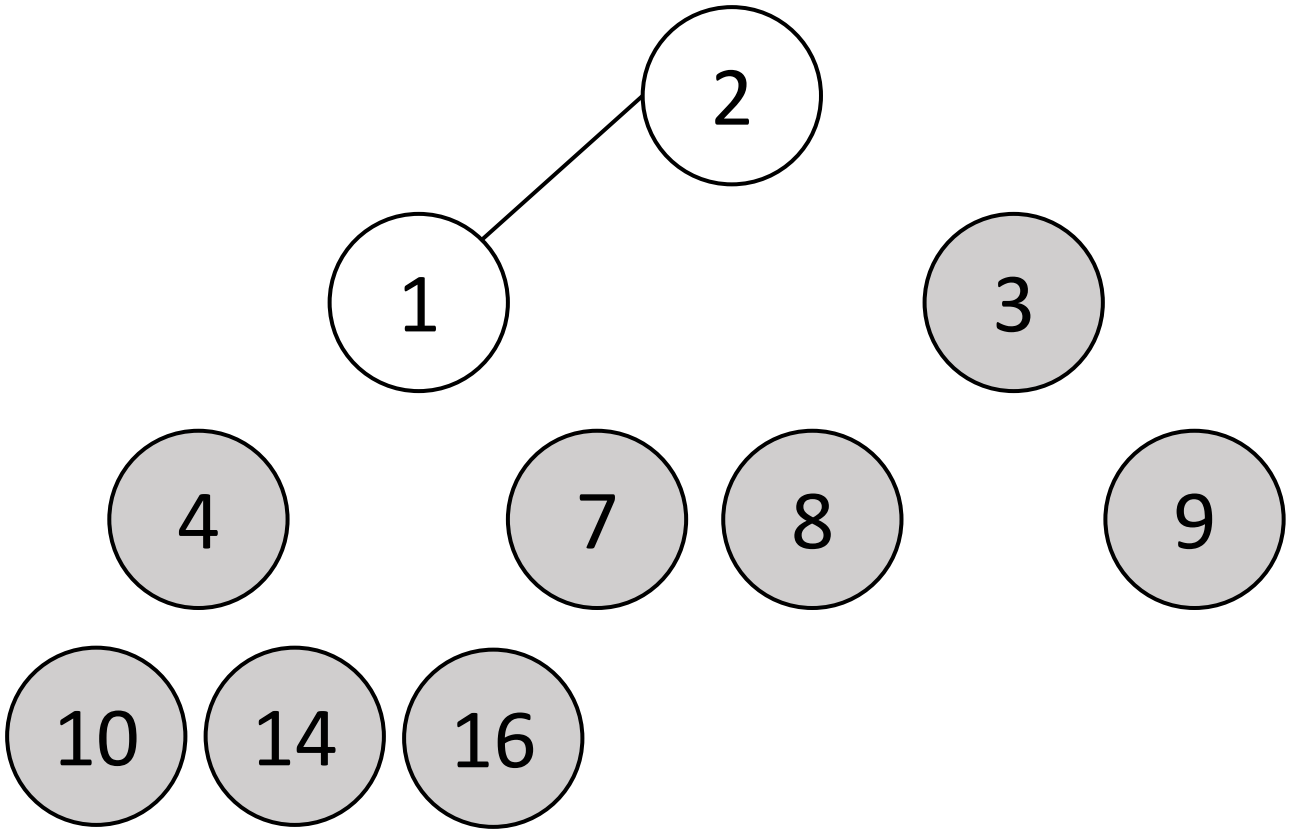
7	4	3	1	2	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10



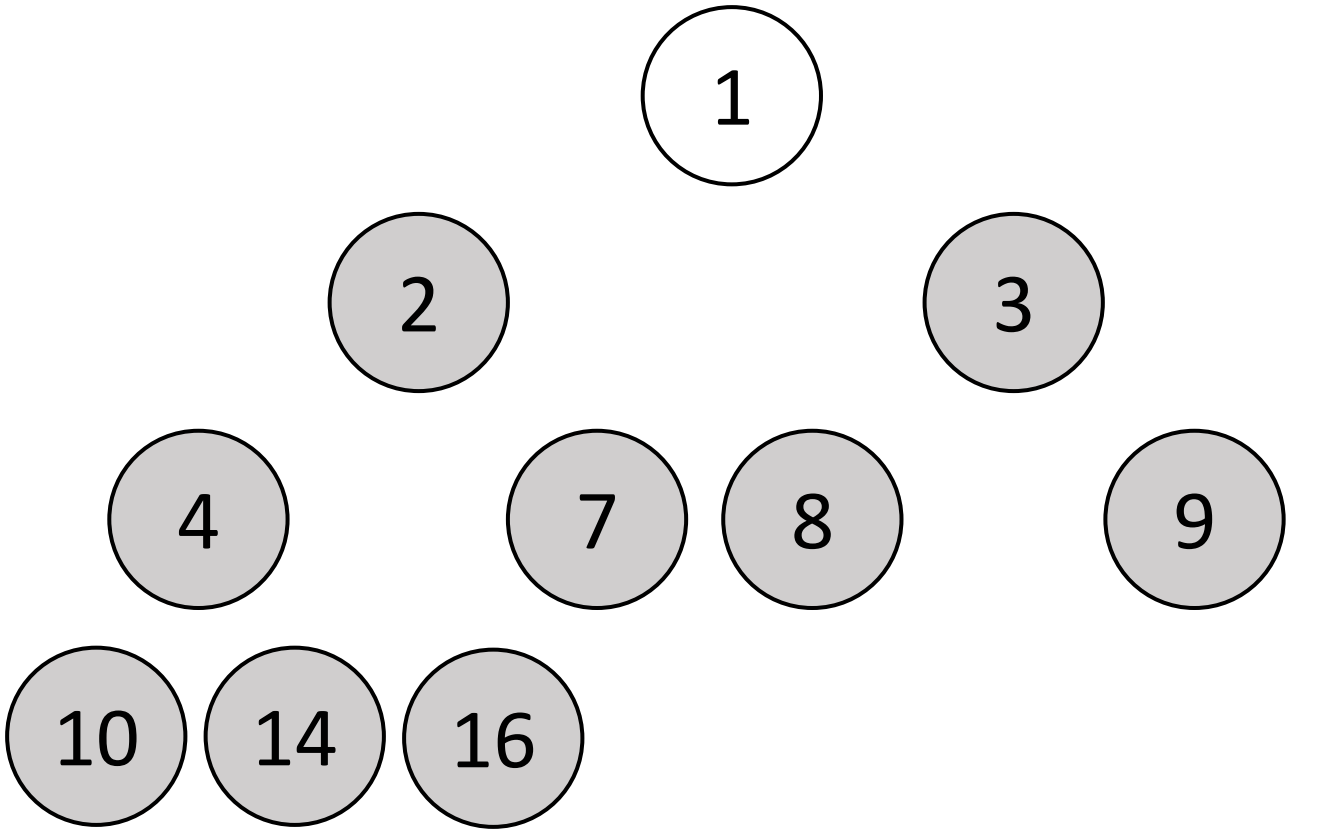
4	2	3	1	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10



3	2	1	4	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10



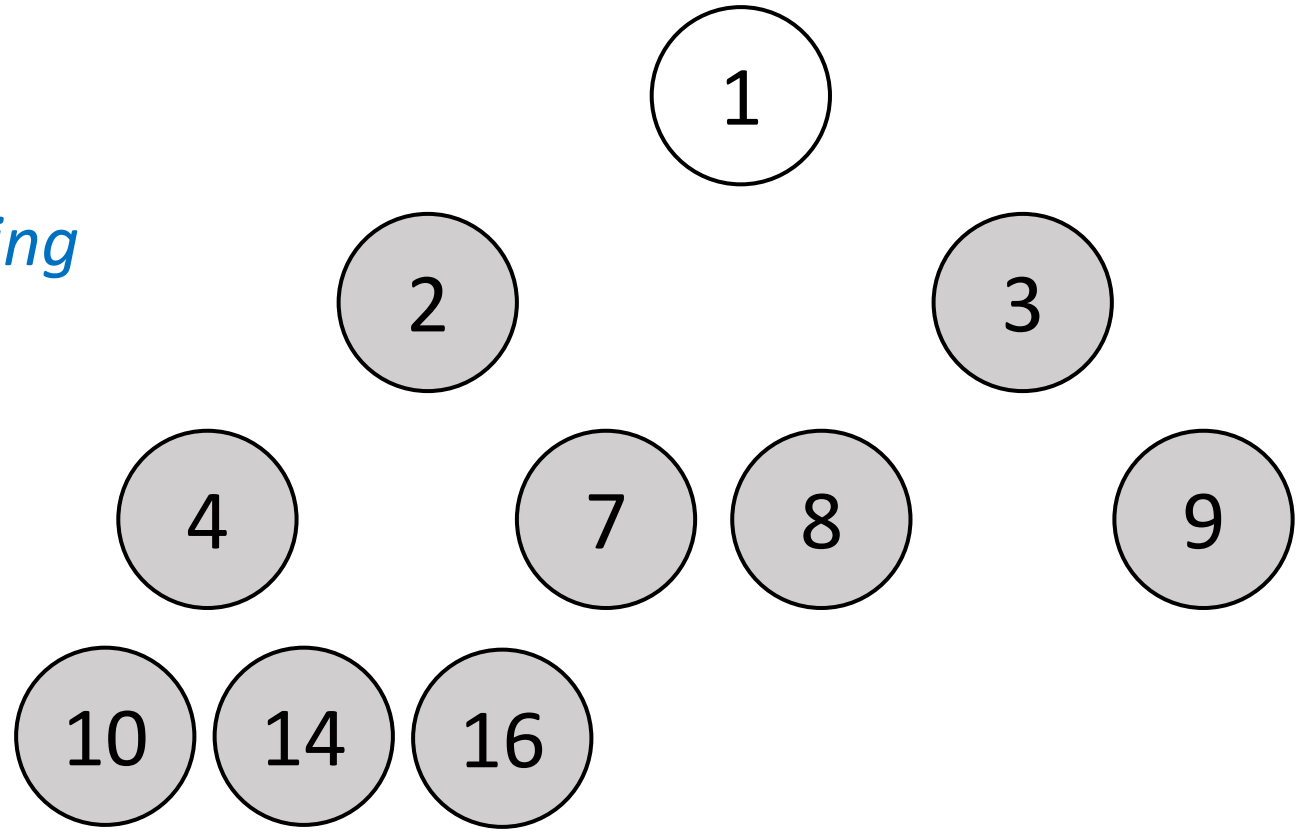
2	1	3	4	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10



1	2	3	4	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10

HeapSort doesn't require any additional space

'In-place' sorting



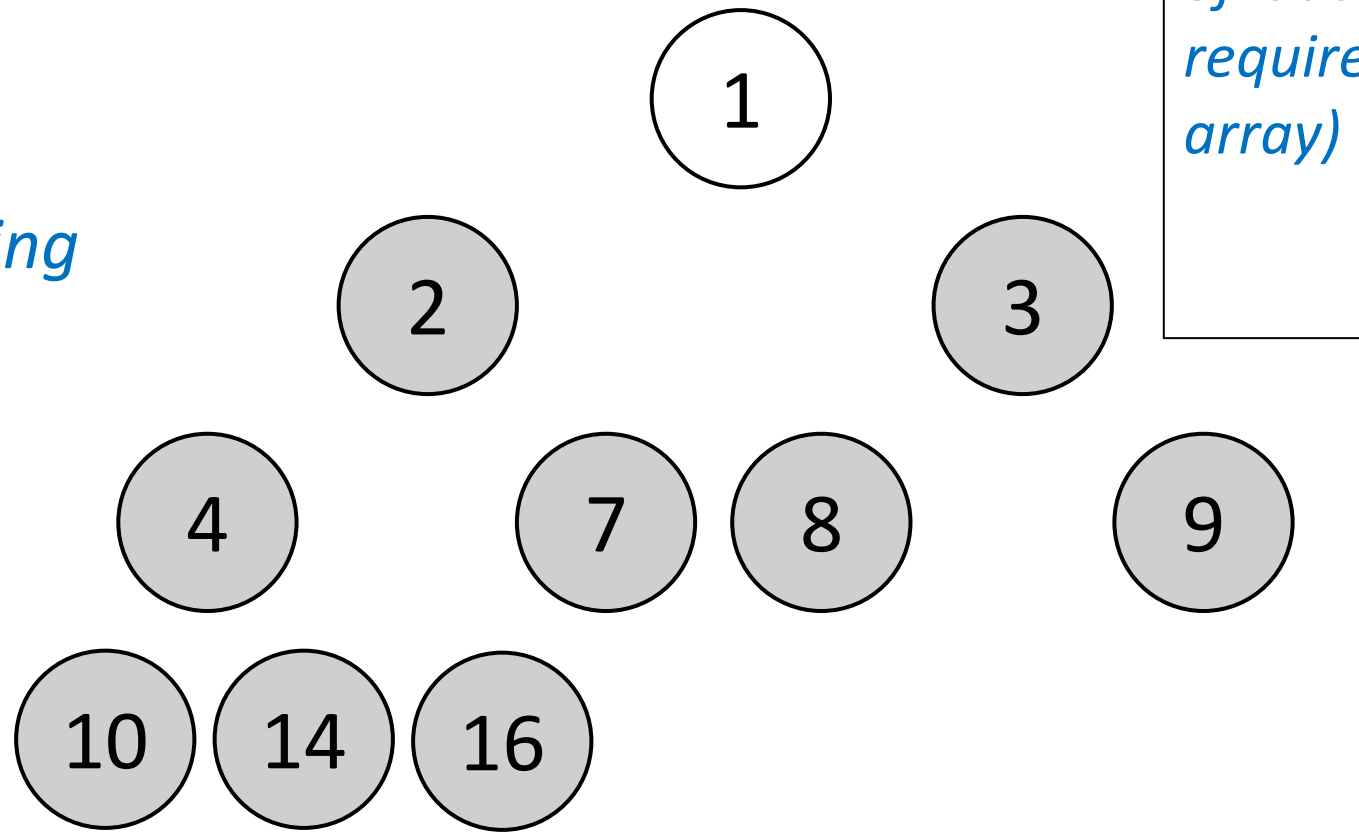
1	2	3	4	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10

HeapSort doesn't require any additional space

'In-place' sorting

Space complexity: amount of 'additional space' required (excluding input array)

O(1) for HeapSort



1	2	3	4	7	8	9	10	14	16
1	2	3	4	5	6	7	8	9	10

```
heapsort(){



}
```

```
heapsort(){
    build_max_heap(heap)



}
```

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2




}
```

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2
        exchange heap[1] with heap[i]


}
```

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1

}
```

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

What's wrong if we swap these two lines?

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

What's wrong if we swap these two lines?

Heapify will access the unwanted location of heap[i]

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

Extracting max/min from heap

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

Don’t forget to re-initiate heapSize back to its original value after heapSort

```
heapsort(){
     build_max_heap(heap)
     for i = heapSize downto 2
          exchange heap[1] with heap[i]
          heapSize = heapSize - 1
          max_heapify(heap,1)
}
```

Time-complexity?

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2
        exchange heap[1] with heap[i]
        heapSize = heapSize - 1
        max_heapify(heap,1)
}
```

$O(logN)$

Time-complexity?

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2
        exchange heap[1] with heap[i]
        heapSize = heapSize - 1
        max_heapify(heap,1)
}
```

Repeated N times

$O(logN)$

Time-complexity?

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2
        exchange heap[1] with heap[i]
        heapSize = heapSize - 1
        max_heapify(heap,1)
}
```

$O(N)$

Repeated N times

$O(logN)$

Time-complexity?

```
heapsort(){
    build_max_heap(heap)
    for i = heapSize downto 2
        exchange heap[1] with heap[i]
        heapSize = heapSize - 1
        max_heapify(heap,1)
}
```

$O(N)$

Repeated N times

$O(logN)$

Time-complexity?

$$O(N) + N \times O(logN)$$
$$= O(NlogN)$$

Priority Queues

maximum()	Returns element with the largest key

maximum()	Returns element with the largest key
extract_max()	Removes and returns the element with largest key

maximum()	Returns element with the largest key
extract_max()	Removes and returns the element with largest key
max_heap_insert (key)	Insert element x into the priority queue with a certain key

maximum()	Returns element with the largest key
extract_max()	Removes and returns the element with largest key
max_heap_insert (key)	Insert element x into the priority queue with a certain key
increase_key (i, value)	Increases the value of the element x’s key by adding a new value

```
Heap_maximum(){

}
```

```
Heap_maximum(){
    return heap[1]
}
```

Time complexity = O(1)

```
Heap_extrax_max(){




}
```

```
Heap_extrax_max(){
    if heapSize < 1
        underflow




}
```

```
Heap_extrax_max(){
    if heapSize < 1
        underflow
    max = heap[1]



    return max
}
```

```
Heap_extrax_max(){
    if heapSize < 1
        underflow
    max = heap[1]
    heap[1] = heap[heapSize]
    heapSize = heapSize-1

    return max
}
```

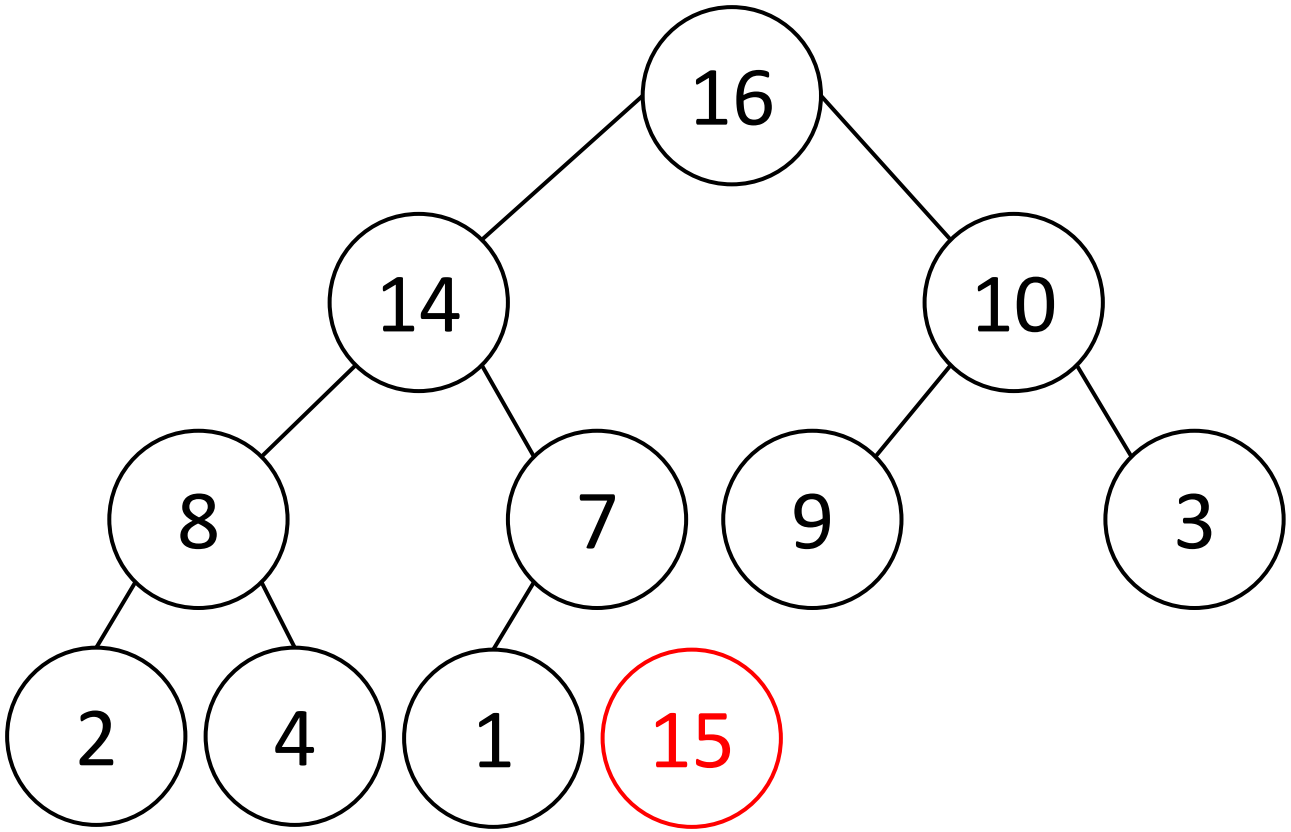
```
Heap_extrax_max(){
    if heapSize < 1
        underflow
    max = heap[1]
    heap[1] = heap[heapSize]
    heapSize = heapSize-1
    max_heapify( ??? )
    return max
}
```

```
Heap_extrax_max(){
    if heapSize < 1
        underflow
    max = heap[1]
    heap[1] = heap[heapSize]
    heapSize = heapSize-1
    max_heapify(1)
    return max
}
```

```
Heap_extrax_max(){
    if heapSize < 1
        underflow
    max = heap[1]
    heap[1] = heap[heapSize]
    heapSize = heapSize-1
    max_heapify(1)
    return max
}
```

Time complexity = $O(logN)$

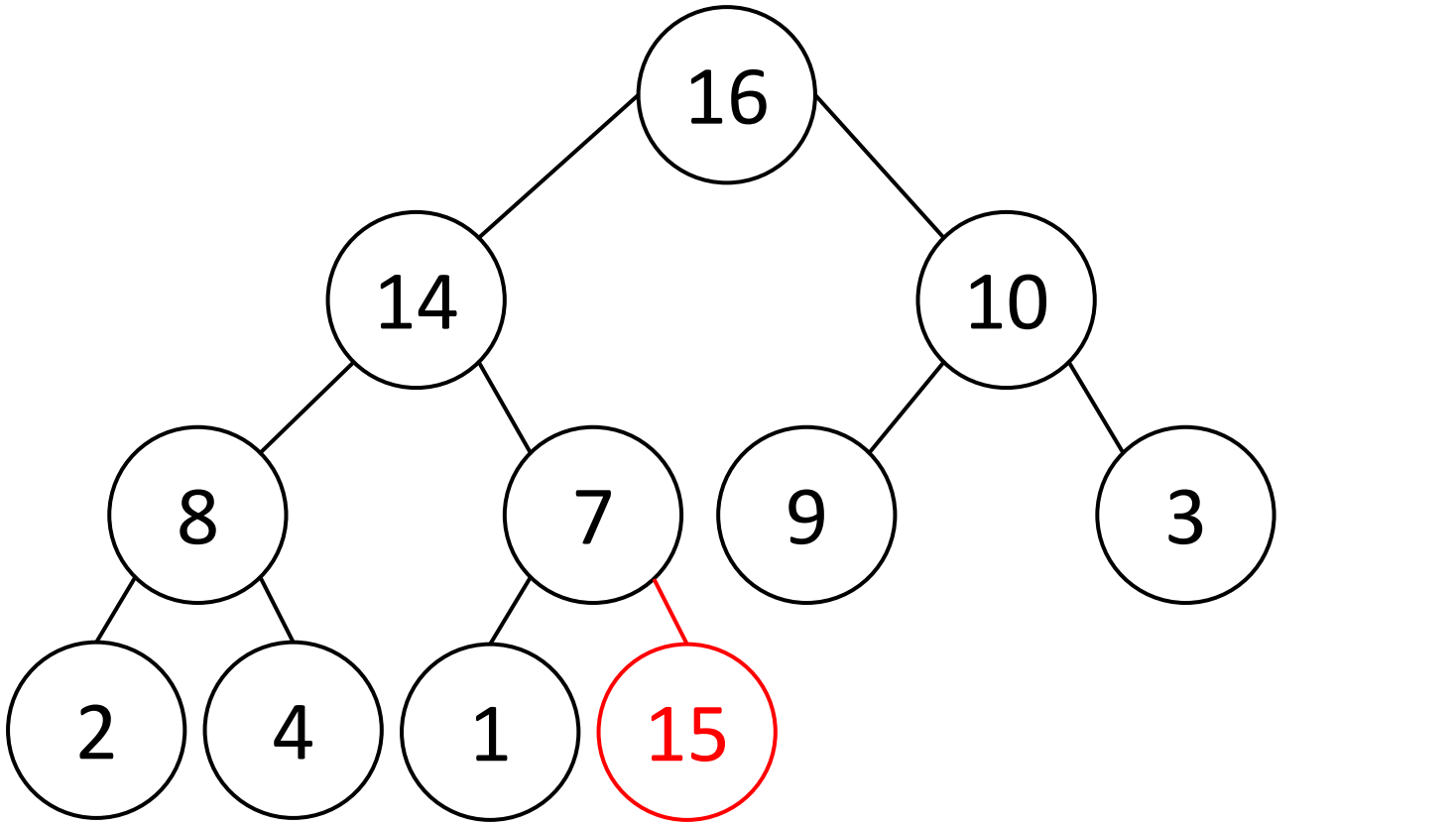
Insert(15) into the heap



16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

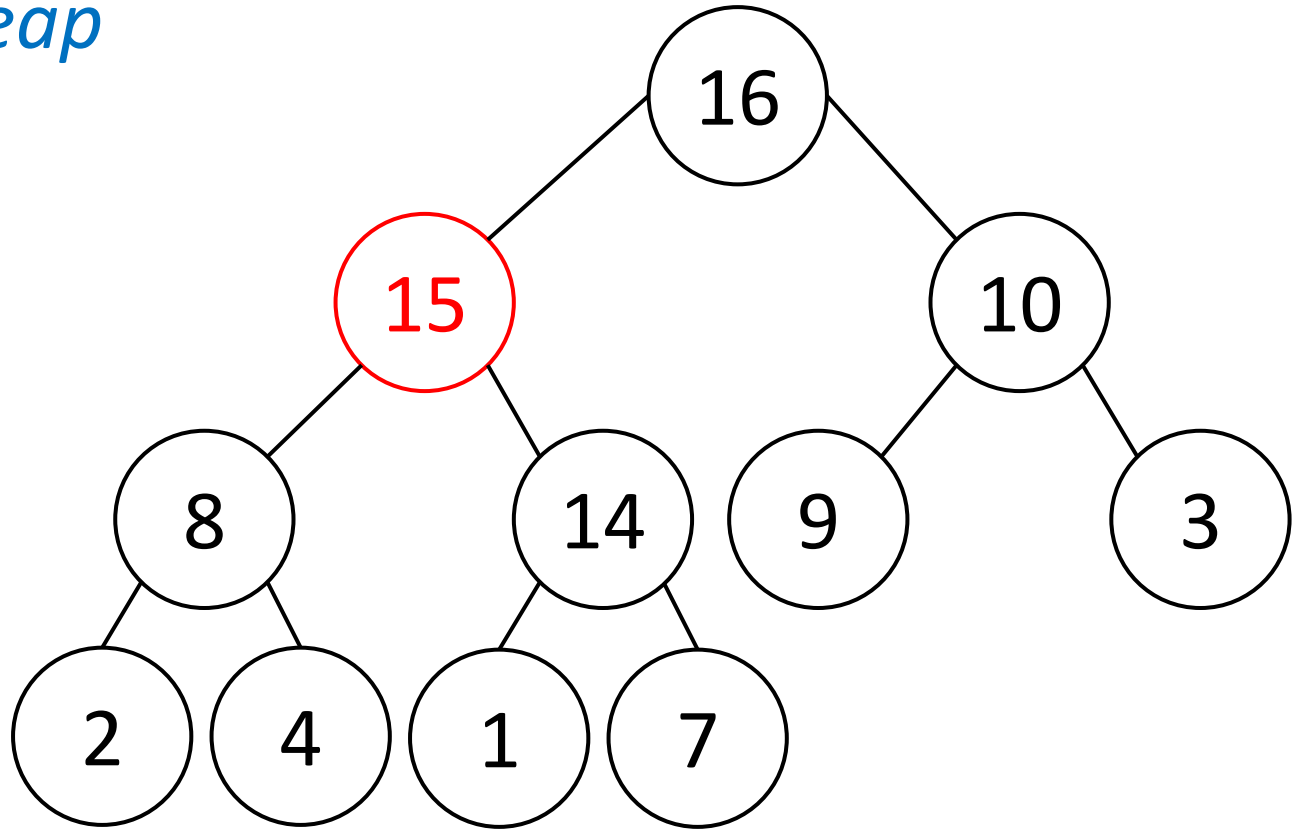
Insert(15) into the heap

Increase heapSize!



16	14	10	8	7	9	3	2	4	1	15
1	2	3	4	5	6	7	8	9	10	11

Status after fixing the heap



16	15	10	8	14	9	3	2	4	1	7
1	2	3	4	5	6	7	8	9	10	11

```
max_heap_insert(key){



}
```

```
max_heap_insert(key){
     heapSize ++
     heap[heapSize] = key



}
```

```
max_heap_insert(key){
     heapSize ++
     heap[heapSize] = key
     while i>1 and heap[parent(i)] < heap[i]


}
```

```
max_heap_insert(key){
     heapSize ++
     heap[heapSize] = key
     while i>1 and heap[parent(i)] < heap[i]
          exchange heap[parent(i)] with heap[i]

}
```

```
max_heap_insert(key){
     heapSize ++
     heap[heapSize] = key
     while i>1 and heap[parent(i)] < heap[i]
          exchange heap[parent(i)] with heap[i]
          i = parent(i)
}
```

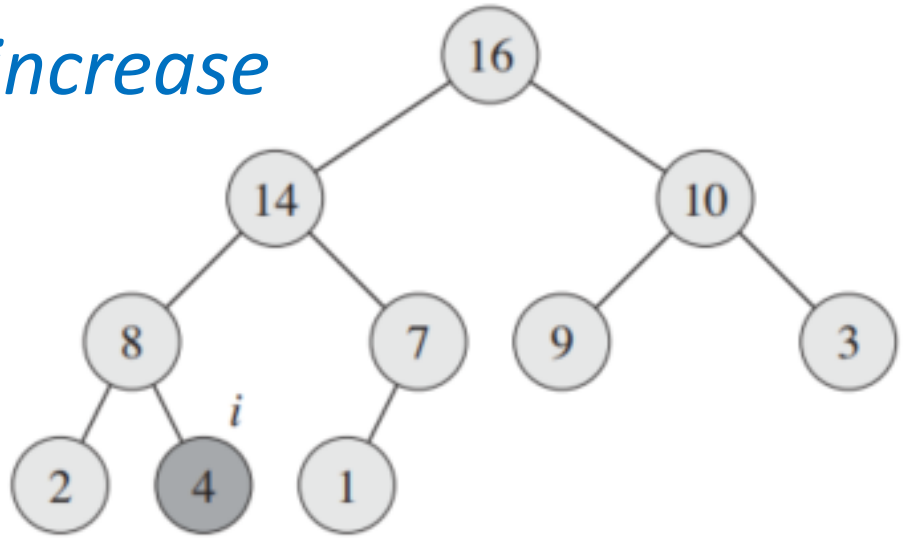
Time complexity?

```
max_heap_insert(key){
     heapSize ++
     heap[heapSize] = key
     while i>1 and heap[parent(i)] < heap[i]
          exchange heap[parent(i)] with heap[i]
          i = parent(i)
}
```

Time complexity?

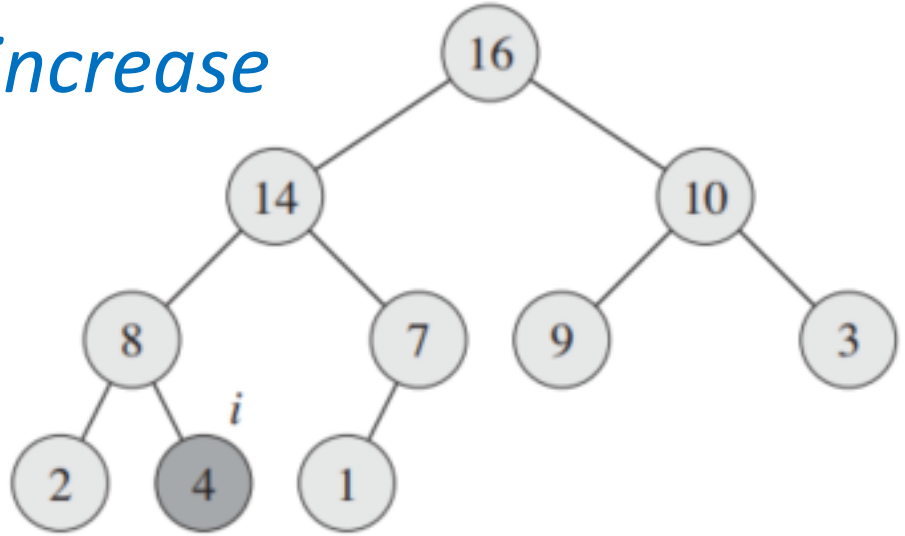
O(logN)

Heap increase key

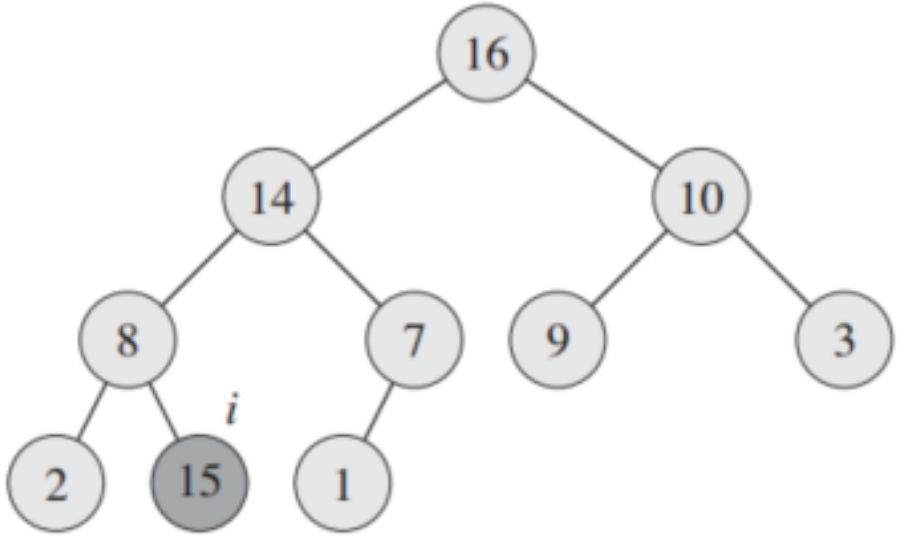


(a)

Heap increase key

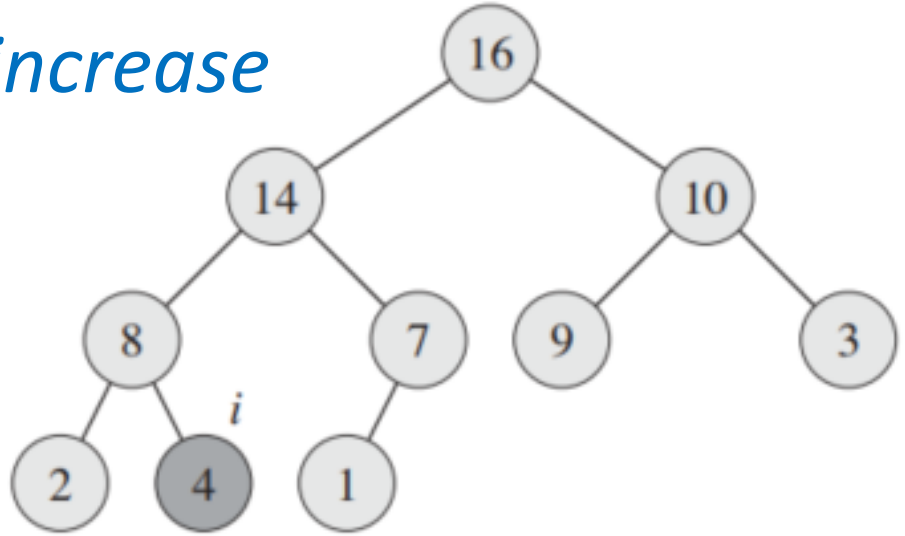


(a)

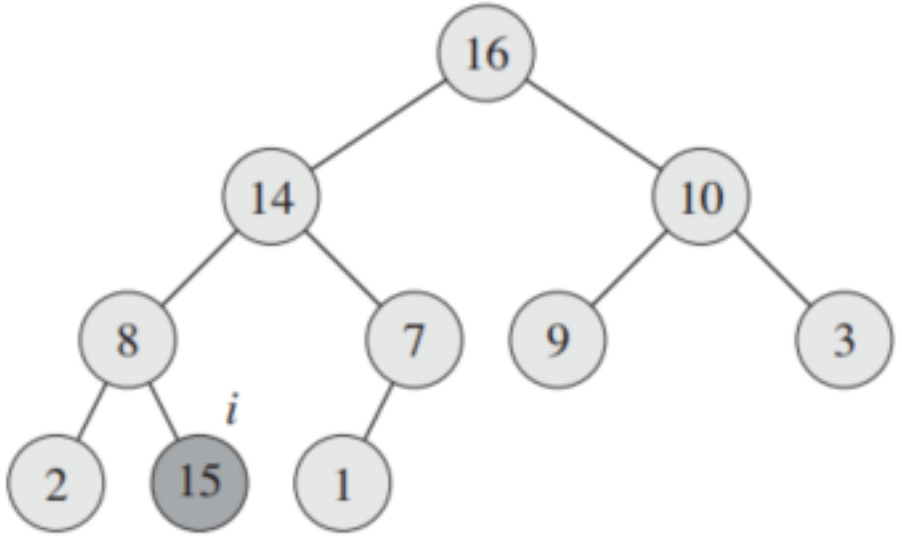


(b)

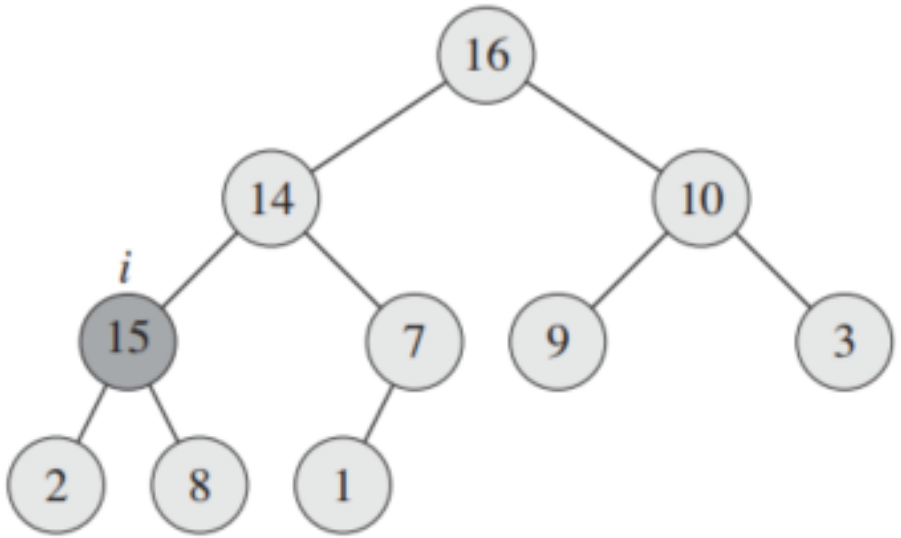
Heap increase key



(a)

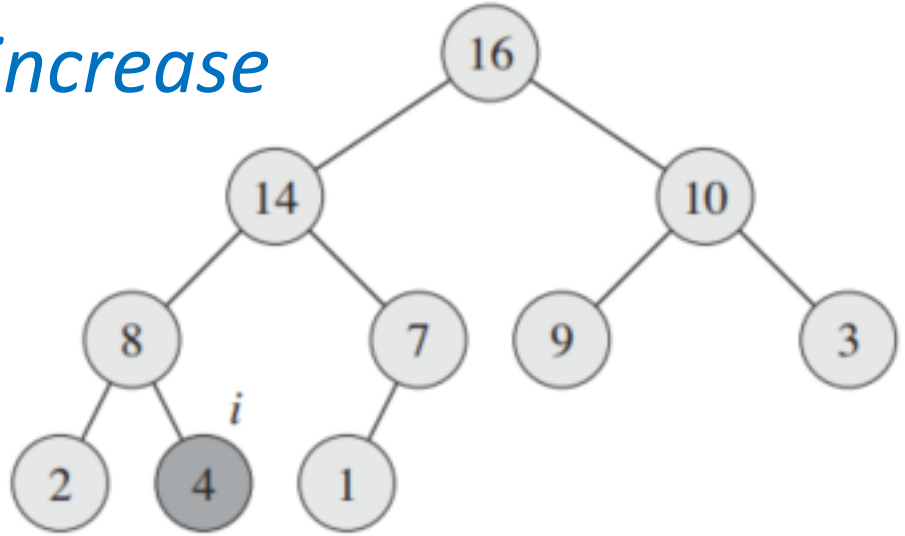


(b)

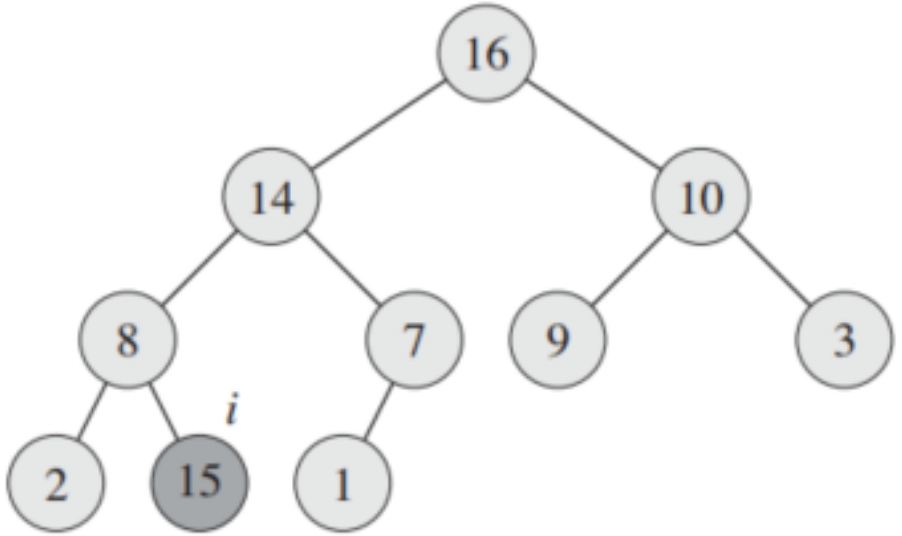


(c)

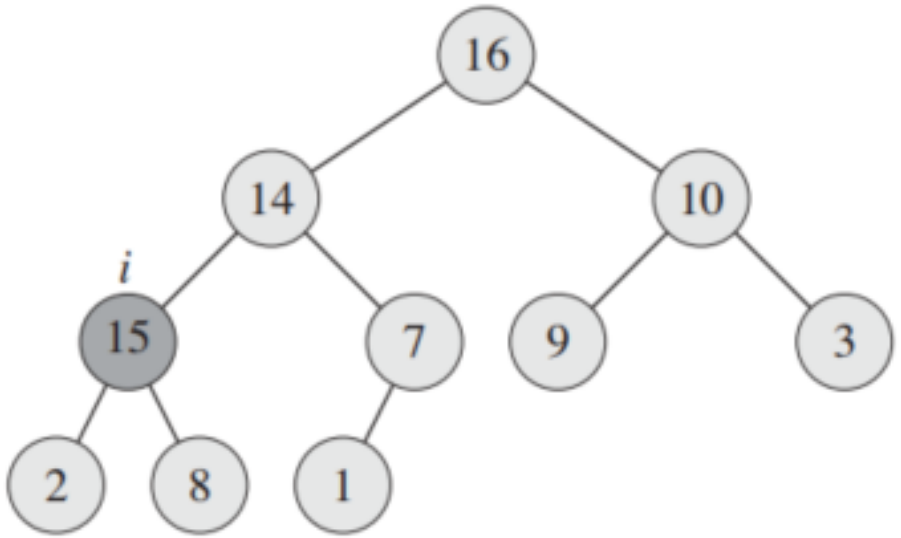
Heap increase key



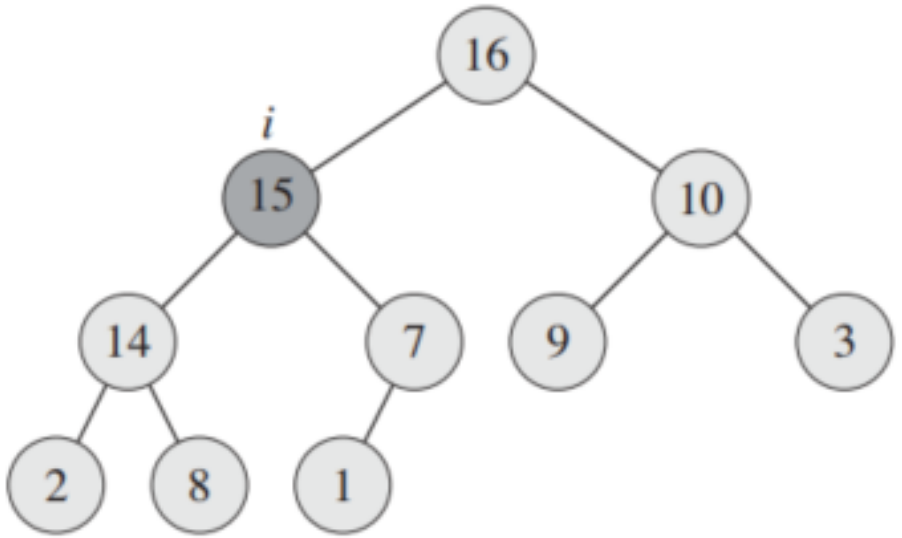
(a)



(b)



(c)



(d)

```
Heap_increase_key(i,value){



}
```

```
Heap_increase_key(i,value){
    heap[i] += value



}
```

```
Heap_increase_key(i,value){
     heap[i] += value
     while i>1 and heap[parent(i)] < heap[i]


}
```

```
Heap_increase_key(i,value){
     heap[i] += value
     while i>1 and heap[parent(i)] < heap[i]
          exchange heap[parent(i)] with heap[i]
          i = parent(i)
}
```

```
Heap_increase_key(i,value){
     heap[i] += value
     while i>1 and heap[parent(i)] < heap[i]
          exchange heap[parent(i)] with heap[i]
          i = parent(i)
}
```

Time complexity?

O(logN)

Meme alert!


MY PROFESSOR STORED ALL THE CLASS'S GRADES
IN A HEAP, AND I FOUND MINE AT THE HEAD


MY PROFESSOR STORED ALL THE CLASS'S GRADES
IN A HEAP, AND I FOUND MINE AT THE HEAD
IT WAS A MIN-HEAP
imgflip.com

References

1. Chapter 6, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. Introduction to Algorithms, Third Edition (3rd. ed.). The MIT Press.
2. Wikipedia contributors. (2024b, June 20). *Binary tree* - Wikipedia. https://en.wikipedia.org/wiki/Binary_tree

Thank you